TRAILLINK: HYBRID COMMUNICATION & SAFETY SYSTEM

INDIVIDUAL THESIS

Vitharana G.G

IT22894038

Individual component: Voice and Push-to-Talk (PTT) Communication

Group ID: R26-IT-135

Report submitted in partial fulfilment of the requirements for the
Bachelor of Science (Hons) in Information Technology
Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this is my own work and this thesis does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my thesis, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Academic integrity and artificial intelligence (AI) disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Vitharana G.G

Student ID: IT22894038



Signature: _____ _____ Date: _____10/09/2026_____

Supervisor certification. The above candidate has carried out research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Vishan Jayasinghearachchi		
Co-supervisor: Ms. Sashini Hathurusinghe		

Abstract

Outdoor groups often lose mobile coverage, and typing is impractical when walking or wearing gloves. TrailLink is an Android-first Flutter application that separates internet-based online trips from nearby-device offline trips built on Google Nearby Connections. This thesis reports its voice and push-to-talk (PTT) component, which provides recorded voice notes and an experimental near-live Live Radio mode with explicit speaker-floor control. The gap addressed is that proximity APIs transport bytes but do not supply validated voice-note creation, replayable playback, floor arbitration, idempotent audio cleanup, or truthful state for non-technical users.

A design-science approach was used. Requirements were traced from the project specification; the component was designed and inspected at a fixed source baseline; and evaluation combines source analysis, an analytical payload-budget model, an automated test matrix, and multi-device and usability protocols. Source analysis shows that notes are validated (minimum 650 ms and 128 bytes) before a database row exists, and that Live Radio follows a seven-state machine whose release path is idempotent and independent of the Nearby session. The analysis also shows that the 250 kilobyte (KB) clip policy is incompatible with Nearby's 32 KB byte-payload limit when audio is Base64-encoded in JavaScript Object Notation (JSON): only about 24 KB of raw audio, roughly 1.5 to 6 seconds at 128 to 32 kbps, fits one payload. Chunked or file-payload transfer is therefore recommended. Physical delivery and user results are reported separately from source evidence. The contribution is a documented, testable voice-and-floor design and an honest account of the limits of commodity proximity audio.

Keywords: push-to-talk, voice notes, Nearby Connections, floor control, offline communication

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to everyone who supported me throughout this research and the preparation of this thesis.

First and foremost, I thank my supervisor, Mr. Vishan Jayasinghearachchi, and my co-supervisor, Ms. Sashini Hathurusinghe, for their continuous guidance, constructive criticism and patience. Their advice on the research question, the evaluation methods and the structure of this thesis shaped the work at every stage.

I am grateful to the Faculty of Computing and the Department of Information Technology at the Sri Lanka Institute of Information Technology for the academic environment, test facilities and guidelines. I also thank the module coordinator, Mr. Nelum Amarasena, the panel members who reviewed my proposal and progress presentations, and the lecturers and staff who helped with teaching, equipment and scheduling.

I sincerely thank my group members, Thanujana D.O (Connectivity and Mesh Intelligence), Pabodhana D.M.D.I (Communication Core) and Dulsara H.G.N (Location, Emergency SOS, Mapping and Syncing). The voice and push-to-talk component depends on the packet router, the Nearby links and the safety features that they built, and our design discussions and code reviews improved every part of the system.

I thank the participants who took part in the usability evaluation and the friends and classmates who lent their phones for multi-device testing.

I also acknowledge the open-source and standards communities whose work this project builds on. I used a generative AI assistant to help structure and draft parts of this thesis, as disclosed in Appendix B; I verified the content and remain responsible for all statements.

Finally, I thank my family and friends for their understanding, encouragement and support.

TABLE OF CONTENTS

Declaration and Supervisor Certification ..... **i**
Abstract ..... **ii**
Acknowledgements ..... **iii**
Table of Contents ..... **iv**
List of Figures ..... **vi**
List of Tables ..... **vii**
List of Abbreviations ..... **ix**
List of Appendices ..... **xi**
1 INTRODUCTION **1**
1.1 Overall Project Context 1
1.2 Individual Component Context 3
1.3 Problem Statement 5
1.4 Research Gap 7
1.5 Aim and Objectives 9
1.6 Research Questions and Hypotheses 10
1.7 Scope, Limitations and Assumptions 11
1.8 Individual Contribution 12
1.9 Thesis Structure 13
2 LITERATURE REVIEW AND RELATED WORK **14**
2.1 Review Method and Scope 14
2.2 Domain and Theoretical Foundations 14
2.3 Related Research 17
2.4 Existing Systems, Products and Techniques 18
2.5 Gap Synthesis and Conceptual Framework 20
3 RESEARCH METHODOLOGY **22**
3.1 Research Design 22
3.2 Data, Participants and Study Materials 24
3.3 Experimental and Study Procedure 25
3.4 Measures and Evaluation Metrics 28
3.5 Data Analysis 29
3.6 Ethics, Privacy, Security and Responsible AI 30
3.7 Reproducibility and Research Artefacts 31
4 PRODUCT AND SOLUTION DESIGN **32**
4.1 Requirements 32
4.2 Component Architecture Within the Complete Solution 34
4.3 Design Alternatives and Decisions 35
4.4 Detailed Design 36
4.5 Standards and Professional Practice 45
5 IMPLEMENTATION, INTEGRATION AND TESTING **46**
5.1 Development Environment and Technology Stack 46

5.2 Component Implementation 46
5.3 Integration with the Other Components 50
5.4 Verification and Software Testing 50
5.5 Deployment and Operation 52
6 RESULTS AND VALIDATION 54
6.1 Research Results 54
6.2 Product and Technical Results 57
6.3 User or Stakeholder Validation 58
6.4 Integration Results 58
6.5 Summary of Findings 58
6.6 Summary of Each Student's Contribution 59
7 DISCUSSION 60
7.1 Interpretation of Findings 60
7.2 Comparison with Literature Baselines and Existing Solutions 61
7.3 Research Contribution 61
7.4 Product Contribution 61
7.5 Limitations and Threats to Validity 61
7.6 Implications 62
8 COMMERCIALISATION AND SUSTAINABILITY 63
8.1 Value Proposition and Target Users 63
8.2 Market and Competitive Position 63
8.3 Business or Sustainability Model 64
8.4 Scalability, Intellectual Property and Compliance 64
8.5 Contribution to the Sustainable Development Goals 65
8.6 Commercialisation Roadmap and Risks 66
9 CONCLUSION AND FUTURE DIRECTIONS 67
9.1 Conclusion 67
9.2 Achievement of Objectives 67
9.3 Individual Research and Product Contributions 68
9.4 Future Directions 68
9.5 Closing Statement 69
REFERENCES 70
GLOSSARY 72
APPENDICES 73
Appendix A: Research Instruments and Test Catalogue 73
Appendix B: Academic Integrity and AI Use Disclosure 74
Appendix C: Requirement Traceability Matrix 75
Appendix D: Usability Materials 75
Appendix E: Consent Form Template 76
Appendix F: Source File Inventory for the Component 76

LIST OF FIGURES

Figure 2.1: Data rates of the audio options considered 20
Figure 2.2: Conceptual framework of the Voice and PTT component 21
Figure 3.1: Research methodology flow 23
Figure 3.2: Lower 95% Wilson bound versus number of successful trials 30
Figure 4.1: Use case diagram of the Voice and PTT component 33
Figure 4.2: Layered architecture of the Voice and PTT component 35
Figure 4.3: Voice-note lifecycle from press to acknowledgement 37
Figure 4.4: Live Radio state machine 38
Figure 4.5: Floor arbitration and Live Radio exchange between two phones 40
Figure 4.6: Receive path for an incoming voice note 41
Figure 4.7: Offline packet envelope used by voice and radio packets 42
Figure 4.8: Conceptual local data model of the component 42
Figure 4.9: Schematic of the voice notes view and the Live Radio view 43
Figure 6.1: Maximum clip duration that fits one byte payload 55

LIST OF TABLES

Table 1.1: Functional components of the TrailLink project 1
Table 1.2: Online trip and offline trip contexts 3
Table 1.3: Comparison of voice notes and Live Radio 4
Table 1.4: Problem decomposition and traceability 7
Table 1.5: Gap analysis by layer 8
Table 1.6: Measurable success criteria for the objectives 10
Table 1.7: Mapping of research questions, hypotheses, experiments and metrics 11
Table 1.8: Assumptions, risks and mitigations 12
Table 1.9: Individual contribution by area 12
Table 2.1: Summary of key sources and their relevance 17
Table 2.2: Categories of voice communication solution for outdoor groups 18
Table 2.3: Audio representation options 19
Table 2.4: From literature to design decisions 21
Table 3.1: Research phases and outputs 22
Table 3.2: Environment record for each physical trial 24
Table 3.3: Experiment protocols 26
Table 3.4: Measures and instruments 28
Table 3.5: Proposed thresholds for interpreting results 28
Table 3.6: Lower 95% Wilson bound when all n trials succeed 29
Table 3.7: Study risks and mitigations 31
Table 4.1: Requirement trace for the Voice and PTT component 32
Table 4.2: Use case specifications 33
Table 4.3: Non-functional requirements relevant to the component 34
Table 4.4: Design alternatives and decisions 35
Table 4.5: Live Radio eligibility conditions 38
Table 4.6: Packet types used by the component 39
Table 4.7: Worked floor-arbitration cases 40
Table 4.8: User-interface states and feedback 43
Table 4.9: Live Radio state transitions 44
Table 4.10: Error conditions and responses 44
Table 5.1: Technology stack for the component 46
Table 5.2: Implementation map 46
Table 5.3: Constants that govern the component 49
Table 5.4: Integration points 50
Table 5.5: Verification plan for the component 51
Table 5.6: Traceability from requirements to tests 51
Table 6.1: Maximum single-payload clip duration versus assumed bitrate 54
Table 6.2: Example clips at an assumed 64 kbps 55

Table 6.3: Audio storage by representation 56
Table 6.4: One-way delay components for Live Radio 56
Table 6.5: Evidence status for each research question..... 57
Table 6.6: Coverage of voice and radio requirements 57
Table 6.7: Contributions of the four group members..... 59
Table 7.1: Threats to validity and mitigations 62
Table 8.1: SWOT analysis of the voice component..... 64
Table 8.2: SWOT analysis (continued)..... 64
Table 8.3: Possible revenue and sustainability options..... 64
Table 8.4: Cost drivers and cost-control measures 64
Table 8.5: Compliance and licensing checklist..... 65
Table 8.6: Contribution to the Sustainable Development Goals..... 66
Table 8.7: Staged roadmap..... 66
Table 9.1: Achievement of objectives..... 68
Table A.1: Voice and PTT test cases 73
Table C.1: Requirement traceability matrix..... 75
Table F.1: Source files owned or used by the component 76

LIST OF ABBREVIATIONS

Abbreviation	Meaning
3GPP	3rd Generation Partnership Project
AAC	Advanced Audio Coding
AAC-LC	Advanced Audio Coding Low Complexity
ACK	Acknowledgement
ADB	Android Debug Bridge
AI	Artificial Intelligence
AODV	Ad hoc On-Demand Distance Vector
API	Application Programming Interface
APK	Android Package
BFCP	Binary Floor Control Protocol
BLE	Bluetooth Low Energy
CI	Confidence Interval
DTN	Delay-Tolerant Networking
FR	Functional Requirement
GPS	Global Positioning System
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
ISO	International Organization for Standardization
ITU-T	International Telecommunication Union Telecommunication Standardization Sector
JSON	JavaScript Object Notation
JWT	JSON Web Token
KB	Kilobyte
MAC	Media Access Control
MCPTT	Mission Critical Push To Talk
NFR	Non-Functional Requirement
P2P	Peer-to-Peer
PCM	Pulse-Code Modulation
PTT	Push-to-Talk
REST	Representational State Transfer
RQ	Research Question
RTT	Round-Trip Time
SDG	Sustainable Development Goal
SDK	Software Development Kit

Abbreviation	Meaning
SQL	Structured Query Language
SUS	System Usability Scale
TTL	Time To Live
UI	User Interface
URL	Uniform Resource Locator

LIST OF APPENDICES

Appendix A: Research Instruments and Test Catalogue 73
Appendix B: Academic Integrity and AI Use Disclosure 74
Appendix C: Requirement Traceability Matrix 75
Appendix D: Usability Materials 75
Appendix E: Consent Form Template 76
Appendix F: Source File Inventory for the Component 76

1 INTRODUCTION

1.1 Overall Project Context

TrailLink is an Android-first Flutter application that supports organised outdoor groups, such as hiking parties, camping groups, field-work teams and disaster-response exercise participants, when internet availability changes during an activity. The application provides two deliberately separate user contexts. An online trip uses authenticated cloud membership, Representational State Transfer (REST) services, MongoDB persistence and Socket.IO real-time messaging. An offline trip uses a locally stored trip with one primary channel, Android Nearby Connections, local queues and channel-scoped packets. The application keeps one selected online trip and one selected offline trip, and changing the operating mode selects the corresponding trip without deleting the other.

This separation is a product decision with a research purpose. It prevents a cloud message from being presented as if every nearby participant received it, and it prevents a nearby message from being silently uploaded into a different cloud membership context. TrailLink is designed for intermittent connectivity and is not a substitute for licensed emergency communication equipment. Nearby communication depends on Android permissions, radio availability, operating-system behaviour, distance, obstacles and the participating devices.

The project is divided into four functional ownership areas, summarised in Table 1.1. These are team responsibilities rather than four separately deployed applications, and a single source file can support more than one component. This thesis reports the third component, Voice and Push-to-Talk (PTT) Communication.

Table 1.1: Functional components of the TrailLink project

Component	**Central responsibility**	**Main hand-over to other components**
Connectivity and Mesh Intelligence	Observe connectivity, manage nearby links, estimate peer quality, apply bounded relay rules	Supplies connection state and packet transport to the core and to voice and safety features
Communication Core	Identity, mode, trip and channel context, chat storage, routing, acknowledgements, online messaging	Dispatches received packets to voice, location and emergency handlers

Component	Central responsibility	Main hand-over to other components
Voice and PTT Communication (this thesis)	Record and play voice notes; acquire and release the talk floor; capture and receive near-live audio	Uses shared Nearby links, local storage and online voice services
Location, Emergency SOS, Mapping and Syncing	Acquire and share location, show maps and freshness, send and acknowledge SOS, retry eligible cloud records	Uses the core context and the available transport

1.1.1 Outdoor communication challenges

Organised outdoor activity creates a recurring coordination problem. Leaders need to keep a headcount, share route changes, warn of hazards, ask for help and confirm that a message was understood, and they must do this while the group is spread along a trail, camp area or work site. Mobile coverage is often weak or absent in valleys, forests, mountains and disaster-affected areas, and it can change within a few hundred metres. The project specification therefore treats connectivity as intermittent rather than simply present or absent.

Two families of solution are commonly used. Internet messaging offers familiar group membership, durable history and rich media, but it fails when coverage fails. Specialised radio and mesh devices work without internet, but need additional hardware, training and a different identity and group-management model [1]. Smartphones already carry Bluetooth and Wi-Fi radios that can exchange data directly with nearby phones, and Google's Nearby Connections exposes this capability to applications [2]. The opportunity addressed by TrailLink is to use these radios inside a complete trip workflow, so that a group of ordinary phones can keep communicating when the internet disappears.

1.1.2 TrailLink operating model

TrailLink keeps online and offline communication in separate trip contexts. Table 1.2 summarises the differences that matter for this thesis. The separation is a deliberate design rule, for the reasons given in Section 1.1.

Table 1.2: Online trip and offline trip contexts

Aspect	Online trip	Offline trip
Membership and identity	Authenticated cloud identity (JavaScript Object Notation (JSON) Web Token (JWT) session), server-side membership checks	Local application identity; shared channel code; validated peer metadata
Transport	REST and Socket.IO through the backend	Nearby Connections byte payloads between connected phones
Durable storage	MongoDB (authoritative) with local cache	SQLite on each phone
Selected context	active_online_trip_id	active_offline_trip_id
Voice notes	Uploaded through the voice application programming interface (API); server voice ID and uniform resource locator (URL) stored	voice_note packet, acknowledged by voice_ack
Floor control	Socket events handled by ptt.socket.js	ptt_request and ptt_release packets with local tie-break
Live Radio	Not offered as an online service in the inspected design	Experimental near-live audio, offline mode only

1.1.3 System-level goals

The specification sets several goals that shape every component. These are mode separation with mode-scoped trip selection, one primary channel per trip, bounded application-layer relay limited to compact safety packets, persistence of accepted work before it is reported, plain-language user feedback without terms such as advertising or peer-to-peer (P2P), and a clear separation between what is implemented and what has been physically proven. The voice component inherits all of them. In particular, because voice and radio audio are excluded from relay, the component must be judged as a direct-link feature.

1.2 Individual Component Context

The Voice and PTT component provides two different spoken-communication experiences and one coordination mechanism. A **voice note** is recorded first, validated, and sent as a completed clip that the receiver can replay. **Live Radio** sends short audio chunks while the user holds a talk control, giving near-live speech between connected phones. **PTT floor control** decides which member is currently allowed to speak, so that a half-duplex conversation does not collapse into overlapping audio.

The component exists in both operating contexts. In an online trip, voice notes are uploaded to the backend and floor events travel through an authenticated Socket.IO channel. In an offline trip, voice notes, floor requests and audio chunks travel as application packets over established Nearby Connections links. The offline path relies on the Communication Core for identity and the channel context, on the Connectivity component for the live links and peer quality, and on the Local Data component for durable storage and permission checks.

Two scope boundaries are important from the start. First, a recorded voice note is not continuous streaming, and floor arbitration is not a distributed-consensus guarantee; the code coordinates local and peer requests and timeouts under ordinary connected conditions. Second, voice notes and Live Radio are direct-peer features only. The application-layer relay of the project forwards approved compact packets (acknowledgements, SOS, location and heartbeat, plus offline text through its own path) but does not forward voice or radio audio.

1.2.1 Why voice matters in the field

Speech has practical advantages outdoors. It is faster than typing, can be produced while walking, does not require the user to look at the screen for long, and carries urgency and tone that short text may not. It also helps users who find typing difficult with gloves or in rain. This is why guides, field teams and responders have long relied on push-to-talk radio. Speech has limits as well: background wind and noise reduce intelligibility, speaking aloud reduces privacy, and a clip cannot be skimmed as quickly as text. The component therefore complements, and does not replace, text and safety messages.

1.2.2 Voice notes, Live Radio and floor control compared

Table 1.3 presents the comparison of voice notes and Live Radio.

Table 1.3: Comparison of voice notes and Live Radio

Property	Voice note	Live Radio
Interaction	Record, release, send as a unit	Hold to talk; audio is sent while held
Delay tolerance	High: replayed later	Low: needs a live link

Property	Voice note	Live Radio
Audio form	Advanced Audio Coding Low Complexity (AAC-LC) clip in an .m4a file	PCM16, 16 kHz, 200 ms chunks
Offline transport	Single voice_note packet (Base64 in JSON)	live_audio_start, chunk and end packets
Receiver behaviour	Stores note, replays on demand, sends voice_ack	Buffers and plays stream, stops on end or 3 s idle
Floor needed	Yes, for recording on the shared channel	Yes, strictly one speaker
Main limitation	Payload size	Bandwidth, latency and jitter
Status	Supported feature	Experimental, behind a setting

Floor control is common to both. It is the rule that decides who may speak, so that two simultaneous speakers do not overwrite each other. In the offline case there is no server to grant the floor, so the phones exchange requests and apply the same deterministic comparison.

1.2.3 Relationship to the other components

The component does not stand alone. Identity, mode and channel context come from the Communication Core, link availability and quality come from Connectivity and Mesh Intelligence, and persistence and permission checks come from the Local Data and Android Services layer. The fourth component can interrupt a live session: the viva guide notes that Live Radio may be released when the emergency interruption path is invoked, so audio and safety interaction should be demonstrated rather than assumed. These dependencies are shown in Figure 4.2 and Table 5.4 and are tested in the integration scenario of Section 6.4.

1.3 Problem Statement

During hiking, camping or field operations a person is often walking, carrying equipment, wearing gloves, or watching the terrain. Typing a message in these conditions is slow and unsafe, whereas speaking is quick and keeps the eyes free. When mobile coverage fails, internet-based voice services stop working. Nearby phone-to-phone transport can continue to work, but a transport API alone does not turn

raw bytes into a dependable voice experience. Six concrete problems must be solved for voice communication in an offline outdoor group.

1. **Capture validity.** Accidental taps produce empty or very short recordings, and a control released twice can create duplicate notes. An empty bubble in the conversation misleads users and wastes storage and airtime.
2. **Transport size.** The platform limits a single Nearby byte payload to 32 kilobytes (KB), while the component policy allows clips of roughly 250 KB and encodes audio as Base64 text inside a JSON packet. It is far from obvious that the clips the policy allows can actually be sent.
3. **Floor contention.** In a group without a central server, two members may press to talk at nearly the same moment. A deterministic rule is needed so that exactly one speaker wins and no stale lock blocks the next speaker.
4. **Resource lifecycle.** Microphone capture, playback and the floor state must be released after errors, quick release, navigation, backgrounding, peer loss and timeouts. Cleanup of audio must not tear down the underlying Nearby session that carries all other group communication.
5. **Truthful state.** A message must not be shown as delivered because a send call returned, and Live Radio must not show "active" after resources have stopped or "free" while a remote lock is valid.
6. **Comprehension.** Users in the field must understand which mode, trip and speaker state apply, without knowing terms such as advertising, discovery or P2P.

The problem addressed by this thesis is therefore: how can ordinary Android phones provide understandable, validated and resource-safe voice notes and push-to-talk communication within a channel-scoped offline trip, and what limits does the commodity transport impose on that experience?

1.3.1 Problem decomposition

Table 1.4 shows the problem decomposition and traceability.

Table 1.4: Problem decomposition and traceability

No.	Problem	Consequence if unsolved	Requirement	Objective
P1	Capture validity	Empty or duplicate notes mislead users and waste airtime	FR-VOICE-001, FR-VOICE-002	O2
P2	Transport size	Allowed clips cannot be delivered; silent failure risk	FR-OFF-006, FR-OFF-007	O4
P3	Floor contention	Overlapping speech; blocked or stale lock	FR-RADIO-001, FR-RADIO-004	O3
P4	Resource lifecycle	Microphone stays busy; group disconnected on release	FR-RADIO-002, FR-RADIO-003, FR-RADIO-005	O3
P5	Truthful state	False delivered or active status; unsafe trust	FR-OFF-006, NFR-USAB-001	O2, O6
P6	Comprehension	Users cannot operate tools under stress	NFR-USAB-002, NFR-USAB-003	O6

1.3.2 Significance of the problem

The problem matters for three reasons. First, voice is often used when a person is least able to type, so errors in a voice feature have a higher cost than in a convenience feature. Second, a feature that reports success falsely can cause groups to rely on a message that was never received, which is why the specification forbids marking delivery on a local send call alone. Third, the platform limit on payload size is a technical fact that determines what the feature can honestly promise. Documenting it, and showing how to design around it, is useful to other developers of proximity applications.

1.4 Research Gap

Outdoor communication is commonly split between internet-dependent messaging and specialised radio or mesh hardware. Internet push-to-talk services offer familiar group semantics but depend on connectivity. Dedicated radios work without internet but need additional hardware, training and a different identity model. Proximity APIs allow phones to exchange data, yet they do not supply a complete voice workflow. For example, Zello is a widely used push-to-talk application that works over carrier or Wi-Fi connections and therefore needs internet access, while Briar synchronises messages directly between nearby devices over Bluetooth or Wi-Fi when the internet is down but is a secure messenger and not a floor-controlled voice tool.

The gap addressed here lies in the middle layer between the transport and the user. Standardised floor-control work exists for conferencing and for mission-critical cellular PTT [3], [4], but those designs assume a server or network infrastructure. Delay-tolerant networking offers store-carry-forward concepts for disrupted networks [5], [6], but it does not specify the user-facing behaviour of a voice clip or a talk button. The gap is thus the lack of a practical, documented, commodity-Android design that combines (a) validated voice-note creation with replayable and truthful state, (b) serverless floor arbitration with idempotent release, (c) an audio lifecycle isolated from the connection lifecycle, and (d) an honest quantification of what the byte-packet transport can carry.

This is an applied research proposition, not a claim of universal novelty. The project specification states that a formal novelty claim would require a systematic literature review and a comparative product study, and this thesis follows that restraint.

1.4.1 Gap analysis matrix

Table 1.5 sets out the gap analysis by layer.

Table 1.5: Gap analysis by layer

Layer	What exists	What is missing for the outdoor use case	Addressed in
Link	Nearby Connections links and byte, file and stream payloads	Trip-scoped use, application identity and session ownership	Chapters 4 and 5
Floor control	Binary Floor Control Protocol (BFCP) and Mission Critical Push To Talk (MCPTT) floor control with servers	Serverless request and release among phones with idempotent cleanup	Section 4.4.3
Audio form	Codecs such as Advanced Audio Coding (AAC) and Opus	A clip and chunk design that respects the 32 KB payload	Section 6.1
Lifecycle	Platform recording and playback APIs	Validation before persistence; replayable bubble; resource release isolated from the connection	Sections 4.4.1 and 4.4.4
Interface truth	Usability heuristics	Mode, trip, floor and delivery state shown without technical terms	Section 6.3

Layer	What exists	What is missing for the outdoor use case	Addressed in
Evaluation	General delay-tolerant networking (DTN) and ad hoc studies	Repeatable phone experiments with PASS and FAIL reporting	Chapter 3

1.5 Aim and Objectives

Aim. To design, implement and evaluate a voice-note and push-to-talk component for TrailLink that provides validated, replayable, resource-safe voice communication within an offline trip on commodity Android phones, and to characterise the limits of the nearby transport for that purpose.

The objectives are:

- O1 Specify functional and non-functional requirements for voice notes, floor control and Live Radio that trace to the project specification.
- O2 Design a voice-note lifecycle (record, validate, persist, send, acknowledge, replay) that works in the online and offline contexts and avoids empty, duplicate and falsely delivered notes.
- O3 Design and implement a floor-control and Live Radio state machine with deterministic tie-breaking and idempotent cleanup that is isolated from the Nearby session.
- O4 Quantify the size constraints of the byte-packet path for voice clips and live-audio chunks, and recommend a transport change where the constraints are violated.
- O5 Verify the component through automated tests and a defined physical protocol, reporting each result as PASS, FAIL, PARTIAL, BLOCKED or NOT TESTED.
- O6 Evaluate whether users correctly interpret the recording, playback, floor and delivery states shown by the interface.

Table 1.6 summarises the measurable success criteria for the objectives.

Table 1.6: Measurable success criteria for the objectives

Objective	Success criterion	Evidence
O1	Every voice and radio requirement is traced to a design element	Table 4.1, Appendix C
O2	No empty or duplicate note in the E1 trials; delivered state set only after acknowledgement	E1 results; source review
O3	Seven-state machine implemented; Nearby session intact in all E3 cycles	Source review; E3 results
O4	Payload limits computed for clips and chunks; recommendation justified	Tables 6.1 and 6.2; E2 results
O5	Each test case reported with one of five statuses and device evidence	Appendix A; Chapter 6
O6	Task success and System Usability Scale (SUS) measured with consenting participants	Study U1

1.6 Research Questions and Hypotheses

RQ1 To what extent do the capture-validation rules and the release guard prevent empty, too-short and duplicate voice notes?

RQ2 What proportion of voice clips permitted by the component policy can be carried by one Nearby byte payload, and how does this depend on bitrate and duration?

RQ3 Does idempotent Live Radio cleanup keep the underlying Nearby session connected over repeated speaker cycles, quick releases and interruptions?

RQ4 Does the deterministic floor rule give a single consistent winner under near-simultaneous requests?

RQ5 Can participants correctly interpret voice and radio states without technical explanation?

The following hypotheses are proposed. H1: after validation and the release guard, repeated press and release trials create no empty and no duplicate notes. H2: after at least three alternating Live Radio cycles, the Nearby session is still connected in every cycle. H3: a one-packet voice note is feasible only below the payload-derived duration limit computed in Chapter 6, so a considerable share of policy-allowed clips cannot be carried by the current encoding. H4: at least 80% of participants complete the interpretation tasks without assistance. H3 is tested analytically and the others experimentally.

Table 1.7 presents the mapping of research questions, hypotheses, experiments and metrics.

Table 1.7: Mapping of research questions, hypotheses, experiments and metrics

Question	Hypothesis	Objective	Experiment	Primary metric
RQ1	H1	O2	E1	Invalid-note rate
RQ2	H3	O4	Analysis; E2	Payload fit; delivery ratio by length
RQ3	H2	O3	E3, E5	Cleanup success; stop latency
RQ4	(tested descriptively)	O3	E4	Floor agreement
RQ5	H4	O6	U1	Task success; SUS

1.7 Scope, Limitations and Assumptions

In scope: recorded voice notes, voice-note playback and acknowledgement, push-to-talk floor control, Live Radio capture and playback, the online voice API and floor events, the local voice and radio records, and the user-interface states for these features.

Out of scope: multi-hop relay of voice or radio audio, cryptographic authentication of audio packets, codec research, iOS, and any connection to public emergency services. SOS and location belong to the fourth component, and link quality estimation belongs to the first.

Assumptions and limitations:

- The analysis uses the source baseline recorded in the viva guide (branch deploy, commit e82b185, inspected on 31 August 2026). Later changes are not reflected.
- Statements about implementation describe the repository as documented; they are not evidence that every physical scenario has passed. Physical results are reported separately and labelled with their status.
- Phones are assumed to run Android with Google Play services, and the user is assumed to grant microphone and Nearby permissions.
- Live Radio is experimental and is not described as guaranteed professional radio service.

1.7.1 Assumption risks

Table 1.8 shows the assumptions, risks and mitigations.

Table 1.8: Assumptions, risks and mitigations

Assumption	Risk if false	Mitigation
Google Play services present	Nearby features unavailable	Record device model; test only supported phones
Permissions granted	Recording or connection blocked	Distinguish platform block from application failure
Two or more phones in range	Live Radio gate refuses	Gate explains the reason; tests state distance
Source baseline fixed	Later code changes results	Report commit e82b185
Clock differences are modest	Unfair tie-break	Record clock offset in E4; discuss as limitation

1.8 Individual Contribution

The individual contribution covers the design, implementation and evaluation of the artefacts in Table 1.9. Several files are shared with other components, for example the packet router that dispatches voice types to this component, and these dependencies are explained rather than claimed as solely authored.

Table 1.9: Individual contribution by area

Area	Principal artefacts (paths relative to the repository root)
Presentation	lib/features/ptt/presentation/: ptt_screen, ptt_controller, push_to_talk_button, voice_note_bubble, speaker_lock_banner
Voice-note logic	lib/features/ptt/data/: ptt_repository, ptt_audio_service, ptt_packet_service, ptt_api, models/voice_note_model
Floor and radio	ptt_floor_controller, models/ptt_floor_state, live_radio_eligibility_service, live_radio_audio_service, models/live_radio_session_model
Local data	ptt_local_data_source (notes, queues, processed packets, floor events, live sessions)
Cloud	backend/src/socket/ptt.socket.js; modules/voice/voice.routes.js and voice.service.js; models/voiceNote.model.js
Research artefacts	Requirement trace, payload-budget model, test catalogue, experiment and usability protocols (Chapters 3, 4 and 6; Appendix A)

The shared-file boundary is explained so that authorship is not overstated. The central packet router, the local database schema and the mode controller are shared with the other components and are described here only where the voice component depends on them. The individually owned work is the PTT presentation, repository, audio, floor,

packet, radio eligibility and local data source files, the cloud voice and floor modules, and the research artefacts listed in Table 1.9. A summary of all four members' contributions is given in Section 6.6, as required by the dissertation guidelines.

1.9 Thesis Structure

Chapter 2 reviews the foundations and related work. Chapter 3 describes the research methodology. Chapter 4 presents requirements, architecture, design alternatives and detailed design. Chapter 5 describes implementation, integration and testing. Chapter 6 reports results and validation, separating analytical, source-level and physical evidence. Chapter 7 discusses the findings, contributions and threats to validity. Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes with achievement of objectives and future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

This chapter is a structured narrative review, not a systematic review. Sources were chosen to explain the technical concepts that the component depends on: push-to-talk and floor control, digital speech coding, end-to-end delay limits, distributed ordering, delay-tolerant networking, and the Nearby Connections platform. Standards bodies (the Internet Engineering Task Force (IETF), the 3rd Generation Partnership Project (3GPP), the International Telecommunication Union Telecommunication Standardization Sector (ITU-T) and the International Organization for Standardization (ISO)) and the official platform documentation were preferred over secondary sources. Product statements in Section 2.4 are general characterisations of categories of system and are not verified specifications of any product.

Consistent with the project specification, no claim is made that no comparable system exists. A formal novelty claim would require a systematic review with documented search strings, databases and inclusion criteria.

2.2 Domain and Theoretical Foundations

2.2.1 Push-to-talk and floor control

Push-to-talk is a half-duplex communication style in which one participant transmits at a time. A talk control grants temporary permission to speak, and release returns the channel to idle. In conferencing, the Binary Floor Control Protocol (BFCP) defines how participants request, are granted and release a shared resource, including queuing and a chair role [3]. Mission-critical push-to-talk over cellular networks specifies requirements for group calls, floor arbitration, priority and pre-emption [4]. Both rely on a coordinating server or network function. TrailLink has no always-present server in offline mode, so its floor logic is exchanged between peers, which is why the problem is closer to distributed arbitration than to a server-granted floor.

2.2.2 Ordering and arbitration in distributed systems

When two peers request the floor at nearly the same time, a rule must select one winner that all peers can compute identically. Distributed-systems theory shows that ordering

events across processes requires logical or synchronised clocks, and that timestamps from unsynchronised physical clocks can disagree [7]. The component compares request timestamps and breaks equal timestamps by identity ordering. This is deterministic given the same inputs, but wall-clock differences between phones remain a threat to fairness and are treated as a limitation (Section 7.5) and an experiment (E4, Section 3.3).

2.2.3 Speech coding and delay

Speech can be carried as uncompressed pulse-code modulation (PCM) or compressed. Advanced Audio Coding Low Complexity (AAC-LC) is a widely supported lossy codec suited to stored clips [8], whereas the Opus codec is one of the mandatory codecs for interactive WebRTC audio, although compressed coding adds some encoding and decoding delay [9]. ITU-T Recommendation G.114 gives guidance on one-way transmission time for conversational quality, with delays below roughly 150 ms generally acceptable for most applications [10]. The component uses AAC-LC for voice notes and uncompressed PCM16 at 16 kHz in 200 ms chunks for Live Radio, which trades bandwidth for implementation simplicity. The 200 ms chunk size alone is already above the 150 ms guidance before any network delay, so Live Radio is best described as near-live rather than conversational telephony.

2.2.4 Nearby Connections and the payload limit

Nearby Connections lets phones discover each other, connect, and exchange data without internet using Bluetooth, Bluetooth Low Energy (BLE) and Wi-Fi mechanisms chosen by Google Play services [2]. The Cluster strategy supports a many-to-many topology within radio range. After connection, applications exchange payloads of several types. A byte payload is intended for small messages and is limited to 32 KB, whereas file and stream payloads carry larger data [2]. The Nearby software development kit (SDK) does not forward application messages between unconnected endpoints, so any multi-hop behaviour is application-layer logic.

2.2.5 Delay-tolerant networking and routing

Delay-tolerant networking (DTN) addresses networks that lack continuous end-to-end paths, using store-carry-forward behaviour [5], [11]. The Bundle Protocol Version 7

formalises this idea [6]. Mobile ad hoc routing protocols such as Ad hoc On-Demand Distance Vector (AODV) discover routes on demand. TrailLink is informed by the DTN problem domain but implements neither protocol, and the Bluetooth Mesh Profile [12] is likewise not implemented. For this component the relevance is a design boundary: audio is latency-sensitive and bandwidth-heavy, so it is kept to direct links.

2.2.6 Android audio capture and permissions

On Android, recording from the microphone is a dangerous permission that must be requested at runtime, and the user may deny it or revoke it later. The local-data specification therefore models permission state as granted, denied, permanently denied, unavailable or optional, refreshes it when the app resumes, and offers Android Settings when a permission is permanently denied. Microphone and audio behaviour also vary across Android versions and vendors; background restrictions, audio focus and other apps using the microphone can interrupt capture. For this reason the component treats interruption as a normal event and routes every interruption to the same cleanup path.

Recorded audio is stored as a file in an app-owned location, not as a large in-memory object. Large audio is file-backed so that memory use remains bounded, and a missing file marks the note unavailable rather than crashing the timeline. These practices are consistent with the privacy requirement that recordings are sensitive personal data and should be confined to app-owned storage [13].

2.2.7 Human factors of push-to-talk interfaces

A press-and-hold control is simple to learn but easy to misuse. A brief accidental touch can start a recording, a finger sliding off the control can end it early, and a user may not know whether anyone heard them. Nielsen's heuristics of visibility of system status, error prevention and recognition rather than recall [14] translate into concrete rules: show who currently holds the floor, discard empty recordings without creating a bubble, and show waiting and delivered states honestly. ISO 9241-11 defines usability as the effectiveness, efficiency and satisfaction with which users achieve goals in a context of use [15]. The context here includes gloves, motion, noise and

stress, so the study measures task success and help requests rather than preference alone.

2.2.8 Reliability patterns used in the design

Four software patterns recur in the component. A finite state machine makes legal transitions explicit, so that the Live Radio interface cannot show a speaking state while the floor is idle. A guard flag makes release idempotent: a second release call returns without effect. A try and finally structure ensures that cleanup runs after errors. A processed-packet table suppresses duplicates, so a repeated packet cannot create a second note. ISO/IEC 25010, developed with the International Electrotechnical Commission (IEC), describes reliability through faultlessness (maturity in the 2011 edition), availability, fault tolerance and recoverability [16]; the guard, finally and restart-normalisation mechanisms address fault tolerance and recoverability directly.

2.3 Related Research

The literature that most directly informs the component falls into three groups. The first group concerns floor control: BFCP and MCPTT define request, grant, release, queuing and priority [3], [4]. The TrailLink rule is much simpler, because it has no chair, queue or server, and its design should be read as a lightweight adaptation for small groups. The second group concerns delay-tolerant and ad hoc networking [5], [6], which motivates the decision to keep the relay bounded and to exclude continuous audio. The third group concerns usability: Nielsen's heuristics on visibility of system status and error prevention and the ISO 9241-11 definition of usability as effectiveness, efficiency and satisfaction in context motivate the requirement that recording, floor and delivery states are truthful and visible.

Table 2.1 summarises how each major source informs the design and where it falls short for this use case. The table is the basis for the gap synthesis in Section 2.5.

Table 2.1: Summary of key sources and their relevance

Source	Contribution	Use in this thesis	Limitation here
BFCP	Request, grant, release and queue of a shared floor	Vocabulary and concepts for floor control	Assumes a floor control server and chair

Source	Contribution	Use in this thesis	Limitation here
MCPTT stage 1	Group call, floor and priority requirements for critical PTT	Requirements context for arbitration and pre-emption	Assumes cellular infrastructure
Lamport	Ordering of events and limits of physical clocks	Explains tie-break and clock-skew risk	Logical clocks not implemented
ITU-T G.114	One-way delay guidance for conversation quality	Reference for Live Radio latency discussion	Written for telephony networks
Opus	Low-delay codec for interactive audio	Future option to reduce bandwidth	Not used in the inspected component
DTN and Bundle Protocol	Store-carry-forward for disrupted networks	Justifies bounded relay and persistence ideas	Not implemented; voice excluded from relay
AODV	On-demand route discovery in ad hoc networks	Contrast: TrailLink has no route discovery	Not applicable to direct links
Nearby Connections	Links and payload types on Android	Defines transport limits	No floor, trip or voice semantics
Usability standards	Heuristics, definition and a questionnaire	Study design and interface rules	General, not specific to PTT

2.4 Existing Systems, Products and Techniques

Table 2.2 compares categories of existing solution on the characteristics that matter to this component. The entries are general and are intended to locate the design, not to rank products.

Table 2.2: Categories of voice communication solution for outdoor groups

Category	Internet needed	Extra hardware	Floor control	Trip or channel scoping
Internet push-to-talk applications	Yes	No	Usually server-granted	Group or channel on a server
Dedicated handheld radios	No	Yes	Physical channel, no software arbitration	Radio channel only
Offline text or mesh messengers	No	No (some use hardware)	Not applicable (mostly text)	Varies
Raw Nearby Connections sample code	No	No	None	None

Category	Internet needed	Extra hardware	Floor control	Trip or channel scoping
TrailLink Voice and PTT	No (offline) or Yes (online)	No	Peer-exchanged request and release with deterministic tie-break	Active offline trip and primary channel

Techniques for recorded clips and for streaming also differ. Recorded clips are simple, replayable, tolerant of delay, and suit a store-and-send model, but they are limited by payload size. Streaming gives immediacy but needs jitter handling, loss recovery and stricter floor handling, and the de-jitter buffer alone can add noticeable delay [9]. TrailLink supports both, which lets the user choose between reliability and immediacy, and keeps streaming experimental.

2.4.1 Audio coding options

The codec choice sets the data rate that the transport must carry. Table 2.3 compares the three options discussed in this thesis. Opus supports a wide range of bitrates and short frame durations suited to interactive use. The component currently uses uncompressed PCM16 for Live Radio and AAC-LC for stored clips.

Table 2.3: Audio representation options

Option	Compression	Typical use	Implication for TrailLink
PCM16, 16 kHz	None; 256 kbps for mono	Raw capture; simple streaming	Simple but heavy: 6,400 bytes per 200 ms chunk
AAC-LC (.m4a)	Lossy, selectable bitrate	Stored clips; broad platform playback	Good file size; limited by Base64 and the 32 KB payload

Option	Compression	Typical use	Implication for TrailLink
Opus	Lossy; 6 to 510 kbps; frames 2.5 to 60 ms	Interactive voice	Would reduce radio load; needs codec support and integration

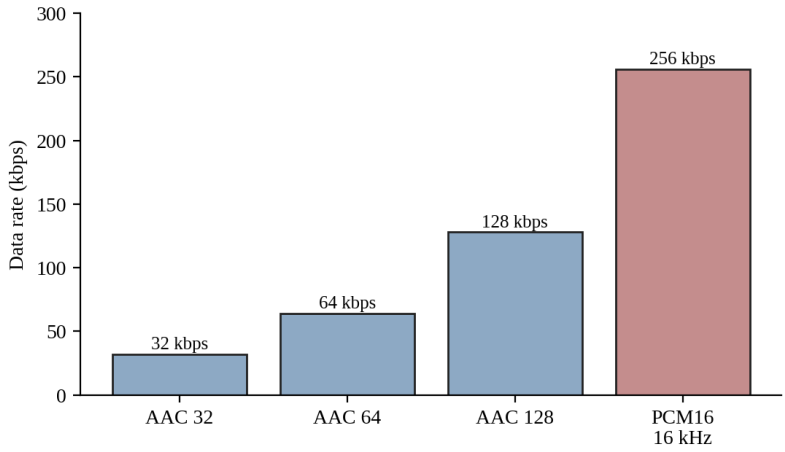


Figure 2.1 sets out the data rates of the audio options considered.

Figure 2.1: Data rates of the audio options considered

2.4.2 Categories of existing solution

Internet push-to-talk applications provide group channels over cellular data or Wi-Fi and usually grant the floor from a server; they are convenient but stop working without connectivity. Handheld licence-exempt radios provide a physical channel and do not need a network, but they require extra hardware, have fixed capabilities and give no trip, identity or delivery record. Offline messengers that use Bluetooth or Wi-Fi between phones, and long-range mesh messengers that use extra radio hardware [1], show that offline group communication is practical, but they focus mainly on text and do not provide a floor-controlled voice feature tied to a trip. Raw Nearby Connections sample code demonstrates the link and payload mechanics only.

2.5 Gap Synthesis and Conceptual Framework

Synthesising the review, the literature offers mature ideas for floor control, speech coding and disruption-tolerant delivery, and the platform offers a link layer, but there is no commodity-phone design that joins them in a trip-scoped, user-comprehensible voice feature with documented limits. Figure 2.2 shows the conceptual framework used in this thesis. Problem conditions, platform limits and user conditions drive five

design principles, which are applied to three feature groups (voice notes, Live Radio and floor control) and evaluated for validity, size budget, cleanup and understanding.

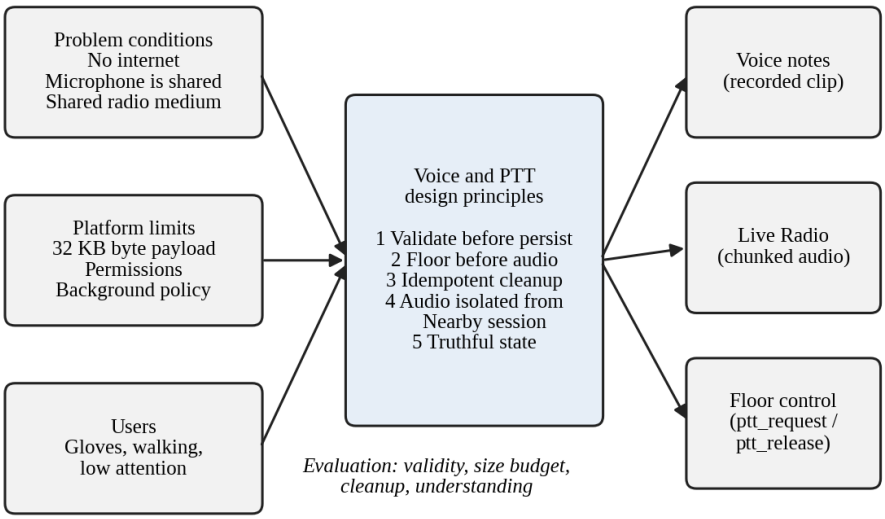


Figure 2.2: Conceptual framework of the Voice and PTT component

Table 2.4 summarises the from literature to design decisions.

Table 2.4: From literature to design decisions

Literature finding	**Design decision**	**Section**
Floor control needs a deterministic rule	Timestamp then identity comparison on both phones	4.4.3
Physical clocks can disagree	Treat skew as a limitation; test in E4	7.5
Interactive audio is delay-sensitive	Call Live Radio near-live and experimental	4.4.4
Byte payloads are limited to 32 KB	Quantify clip budget; recommend chunking or file payload	6.1
DTN and relay add amplification and latency	Keep voice and radio direct-only	4.3
Users need visible status and error prevention	Validate before persist; show floor and delivery state	4.4.1
Reliability needs recoverability	Idempotent cleanup; restart normalisation	5.2.2

In summary, the literature supports each design decision but also shows that no single source offers a ready design for serverless, trip-scoped voice on commodity phones. This confirms that the contribution is an integration and evaluation exercise, as stated in Section 1.4.

3 RESEARCH METHODOLOGY

3.1 Research Design

The study follows a design-science approach [17], [18]: an artefact is built to address a stated problem and is then evaluated against explicit criteria. Because the artefact depends on physical radios and human users, the evaluation is mixed-method. Four kinds of evidence are kept separate throughout the thesis, as required by the project specification, which says that source implementation and physical validation must be reported separately and that unverified behaviour must not be represented as physically proven.

1. **Source evidence:** what the repository at the inspected baseline implements.
2. **Analytical evidence:** quantities derived from documented parameters, such as the payload budget.
3. **Automated-test evidence:** results of deterministic widget, repository and integration tests.
4. **Physical and user evidence:** results of multi-device experiments and the usability study.

Table 3.1 presents the research phases and outputs.

Table 3.1: Research phases and outputs

Phase	Activity	Output
1. Requirements	Extract and trace voice, radio and non-functional requirements from the specification set	Requirement trace (Table 4.1)
2. Design	Compare design alternatives; design lifecycle, state machine and packets	Design decisions (Table 4.4), Figures 4.2 to 4.4
3. Implementation	Implement and inspect the component at the baseline commit	Implementation map (Table 5.2)
4. Analytical study	Compute payload budgets for clips and chunks	Tables 6.1 and 6.2, Figure 6.1
5. Verification	Run automated tests and physical experiments E1 to E5	Result tables (Chapter 6)

Phase	Activity	Output
6. User study	Run task-based usability evaluation with consent	Task success, SUS (Section 6.3)

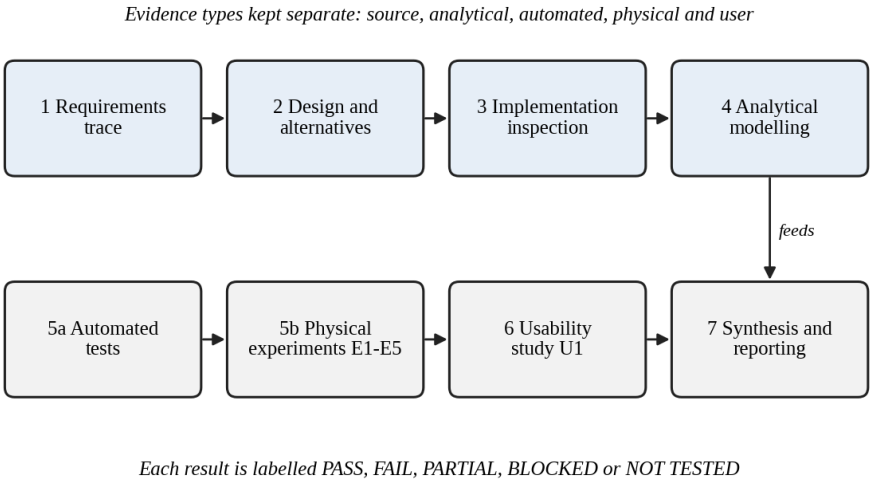


Figure 3.1: Research methodology flow

Figure 3.1 shows the sequence. Phases 1 to 3 establish what the component is supposed to do and what it does at the baseline. Phase 4 derives quantities that do not need hardware. Phases 5 and 6 collect measured evidence. The final phase combines all evidence into findings while keeping the evidence types distinguishable. The approach is appropriate because the artefact is a software component whose behaviour depends on platform radios and human interpretation, so neither a purely analytical nor a purely experimental study would be sufficient.

3.1.1 Justification of the approach

A design-science approach fits because the thesis builds and evaluates an artefact to solve a defined problem rather than testing a theory about a population. The mixed-method evaluation fits because the research questions are of two kinds: technical (RQ1 to RQ4), which are answered with counts, intervals and timings, and human (RQ5), which is answered with task success and a standard questionnaire [19]. Keeping an explicit separation between source, analytical, automated and physical evidence follows the project rule that unverified behaviour must not be described as proven.

3.2 Data, Participants and Study Materials

The study materials are the TrailLink specification set (the system requirements specification and the four component specifications), the viva guide that maps source files to components, and the repository itself at the baseline recorded in the guide. No external dataset is used. Experimental data are generated by the author's test devices and logged locally.

Physical experiments need at least two Android phones for basic acceptance. The device record for each trial shall state model, Android version, application build or commit, permissions, mode, trip and channel, distance, obstacles, and the action performed. The device record is kept in Table 3.2, with one entry per phone and per session. Usability participants are recruited from 8 to 12 undergraduate students of the Faculty of Computing and members of a university hiking or outdoor club, and every participant gives informed consent before the session.

3.2.1 Test environment definition

Each physical trial is recorded in the environment table in Table 3.2 so that results can be reproduced and compared. The same record supports the requirement that evidence identify device models, Android versions, build, permissions, mode, trip and channel, distance, topology and actions.

Table 3.2: Environment record for each physical trial

Field	Example or instruction	Value
Devices and roles	Model of phone A and phone B	Phone A (trip owner) and Phone B (member); model names are entered per session on the data sheet
Android version	Major version and security patch	Android major version and security patch level of each phone, entered per session
Application build	Branch and commit (baseline e82b185 or later)	Branch deploy, commit e82b185 (baseline), or the later commit under test

Field	Example or instruction	Value
Permissions	Microphone, Nearby, location, notifications	Microphone, Nearby/Bluetooth and location granted; notification permission as required
Mode, trip and channel	Offline; trip name; channel code	Offline mode; one Offline Trip with its primary channel code, entered per session
Distance and obstacles	Metres; line of sight or obstructed	Metres from 1 m upwards; line of sight or obstructed, entered per scenario
Battery and background policy	Optimisation settings; battery level	Battery optimisation setting noted; battery level above 50% at the start of each session
Network state	Airplane mode or Wi-Fi off for offline trials	Mobile data off and no Wi-Fi internet connection for offline trials
Date, place and tester	Free text	Entered per session by the tester (the author)

3.2.2 Participants and sampling

Participants for study U1 are recruited by convenience sampling from 8 to 12 undergraduate students of the Faculty of Computing and members of a university hiking or outdoor club. Convenience sampling is acceptable for a formative usability study but limits generalisation. Inclusion criteria are adult age and ownership or regular use of an Android phone. Exclusion criteria are prior involvement in developing TrailLink. A target of 8 to 12 participants is proposed because small formative studies commonly reveal the main usability problems; the number must be justified by the supervisor.

3.3 Experimental and Study Procedure

Five experiments and one usability study are defined (Table 3.3). Each experiment fixes the application build, trip and channel, and repeats the scenario enough times to report a distribution or interval rather than a single anecdote. Test alerts and test audio are labelled so that they cannot be mistaken for real emergencies.

Table 3.3: Experiment protocols

ID	Question	Procedure	Repeats
E1	RQ1 capture validity	On one phone: (a) taps shorter than 650 ms, (b) normal holds, (c) rapid double release, (d) permission denied. Count rows created in the voice-note table.	n >= 30 per case
E2	RQ2 transport size	Between phones A and B send clips of about 1, 2, 3, 5, 8 and 10 s. Record send result, receiver playback and acknowledgement (ACK) latency. Compare with the analytical limit.	n >= 10 per length
E3	RQ3 isolation	Alternate speakers A then B for at least three cycles; include quick release during startup. After each cycle check floor idle and Nearby connected, then send a voice note.	>= 10 sessions
E4	RQ4 floor tie-break	Both phones press within a short window. Record winner on each phone, whether they agree, and whether a stale lock remains.	n >= 30
E5	Interruption	During streaming, background the app, close the route, or move a peer out of range. Record time until the receiver stops playback and floor clears.	n >= 10 per case
U1	RQ5 comprehension	Tasks: identify mode and trip, record and send a voice note, read delivery state, use Live Radio, recognise a peer-held floor. Record success, time and help.	8-12 participants

3.3.1 Detailed protocol for E1 (capture validity)

1. Install the baseline build on one phone and open an offline trip with an active channel.
2. Open the Voice notes view and confirm the microphone permission state.
3. Perform case (a): tap the control for less than about 300 ms, 30 times, and count new note rows.
4. Perform case (b): hold for 2 s and release, 30 times, and count new note rows.
5. Perform case (c): trigger two near-simultaneous release events on the control, 30 times, and count rows.
6. Perform case (d): revoke the microphone permission, attempt to record 10 times, and count rows and messages shown.
7. Export the counts from the database or the user interface (UI) and record them in the results table.

3.3.2 Detailed protocol for E2 (voice-note size sweep)

1. Connect phones A and B in one offline channel and confirm that both show the other as connected.
2. On A, record clips of about 1, 2, 3, 5, 8 and 10 s; keep the speaker and noise constant.
3. For each clip, record the file size on A, the packet size if logged, and the send result.
4. On B, record whether the note appeared, whether it played, and the time to the acknowledgement on A.
5. Repeat each length at least 10 times and log any error message or silent loss.
6. Compare the largest successful length with the analytical limit in Chapter 6.

3.3.3 Detailed protocol for E3, E4 and E5

For **E3**, phone A holds Live Radio for about 5 s and releases, then phone B does the same, and the pair repeats for at least three cycles; in some runs the release is performed immediately after pressing so that it arrives during startup. After every cycle the tester checks that the floor banner shows free, that both phones still show the other as connected, and that a voice note can be sent. For **E4**, the testers press at the same instant on both phones, using a countdown, and record the winner shown on each phone and whether a stale lock remains afterwards. For **E5**, during a live stream the tester performs one interruption: backgrounding the sender app, closing the Live Radio route, or moving the sender out of range. The time until the receiver stops playback and clears the floor is read from timestamped logs.

3.3.4 Protocol for the usability study U1

Each session lasts about 20 minutes. The facilitator explains that the system is under evaluation and not the participant, obtains consent, and gives the participant a pre-configured offline trip and two connected phones. The participant performs the tasks in Appendix A, thinking aloud if comfortable. The observer records success, time, assistance and misunderstood terms, and the participant then completes the SUS questionnaire and answers two open questions about confusing parts and about trust in the delivered state.

3.4 Measures and Evaluation Metrics

Table 3.4 shows the measures and instruments.

Table 3.4: Measures and instruments

Measure	Definition	Instrument
Invalid-note rate	Rows created from empty, short or duplicate release attempts divided by attempts	Database query after E1
Delivery ratio	Voice notes acknowledged by the receiver divided by notes sent	ACK status in database and UI
ACK latency	Time from send to persisted delivered status	Timestamped logs
Payload fit	Serialised packet size compared with 32 KB	Analytical model and logged size
Cleanup success	Cycles ending with floor idle, stream cleared and Nearby connected	Session and floor records
Floor agreement	Trials where both phones agree on the winner	Floor event logs
Stop latency	Time from interruption to receiver playback stop	Logs
Task success and time	Share of tasks completed unaided and seconds taken	Observer sheet
Perceived usability	System Usability Scale score	Questionnaire

3.4.1 Success thresholds

Thresholds make the study falsifiable. They are proposals to be agreed with the supervisor before data collection, not post-hoc targets.

Table 3.5 sets out the proposed thresholds for interpreting results.

Table 3.5: Proposed thresholds for interpreting results

Measure	Proposed threshold	Interpretation
Invalid-note rate (E1)	0 in all trials; lower bound of 95% interval at least 0.88 for n = 30	H1 supported
Cleanup success (E3)	100% of at least 10 sessions	H2 supported
Floor agreement (E4)	100% where both requests were received	Rule is consistent
Delivery by length (E2)	Report largest length with 10 of 10 acknowledged	Defines practical clip limit
Task success (U1)	At least 80% of tasks unaided	H4 supported
SUS (U1)	Mean at or above 68 (commonly cited average)	Acceptable usability

3.5 Data Analysis

Proportions such as delivery ratio and cleanup success are reported with the number of trials and a 95% Wilson score interval [20], because the Wilson interval behaves better than the normal approximation for small samples and for proportions near 0 or 1. With k successes in n trials, p = k/n and z = 1.96:

Latencies are reported as median and inter-quartile range, since they are usually skewed. Chapter 6 also uses the following size relations. Base64 expands n raw bytes to 4 * ceil(n/3) characters. If L is the payload limit, J the JSON envelope overhead, and b the audio bitrate, the largest raw clip that fits in one payload and its maximum duration are:

For Live Radio, the raw chunk size for sample rate f (Hz), 2 bytes per sample and chunk duration T (s) is f * 2 * T bytes. SUS responses are scored by the standard method and reported as a mean with spread.

3.5.1 Sample size and interval width

The Wilson interval shows how many trials are needed to claim reliability. If every one of n trials succeeds, the lower bound of the 95% interval is n divided by the sum of n and z squared, with z = 1.96. Table 3.6 and Figure 3.2 show that 10 flawless trials only support a lower bound of about 0.72, while 30 trials give about 0.89 and 100 trials about 0.96. A single successful demonstration therefore says little about reliability, which is why repeated trials are specified.

Table 3.6: Lower 95% Wilson bound when all n trials succeed

Trials n	Lower bound
5	0.566
10	0.722
20	0.839
30	0.886
50	0.929
100	0.963

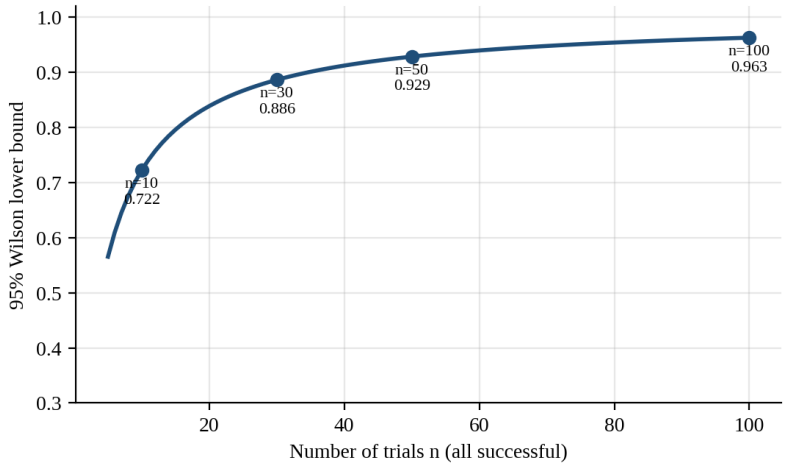


Figure 3.2: Lower 95% Wilson bound versus number of successful trials

3.5.2 Handling of failures and missing data

Failed trials are never dropped. A trial that fails because of a platform block, such as a revoked permission or a vendor battery restriction, is recorded as BLOCKED and explained; a trial that fails because of the application is recorded as FAIL. Trials that were not run are recorded as NOT TESTED. Latency statistics use only trials with a defined acknowledgement, and the number of excluded trials is reported alongside.

3.6 Ethics, Privacy, Security and Responsible AI

Voice recordings and locations are sensitive. The user study obtains informed consent, tells participants that audio is recorded only for the task and deleted afterwards, and stores no personal message content in the results. Experiments do not use real emergencies, and any alert is labelled as a test. In the application, microphone access follows Android runtime permission rules, and recording paths are constrained to app-owned storage. Where the project processes personal data, the Sri Lankan Personal Data Protection Act [21] is considered in Chapter 8. Generative AI was used as a drafting and review aid for this thesis and is disclosed in Appendix B; the author remains responsible for accuracy and conclusions.

Table 3.7 summarises the study risks and mitigations.

Table 3.7: Study risks and mitigations

Risk	Mitigation
Participants feel evaluated or embarrassed	Explain that the system is evaluated; allow withdrawal at any time
Recorded voice is personal data	Record only during tasks; store on the test phone; delete after analysis; no names in results
Test audio mistaken for a real alarm	Label tests clearly; no real emergency scenario
Outdoor trial safety	Test in safe public areas; no hazardous terrain; second person present
Data leakage of locations or routes	Do not log exact routes in the thesis; aggregate only
Over-claiming from small samples	Report intervals, counts and limitations

3.7 Reproducibility and Research Artefacts

Reproducibility rests on a fixed source baseline (branch deploy, commit e82b185), the specification documents, the test catalogue in Appendix A, and a device log for every physical trial. During local device development the backend is reached by Android Debug Bridge (ADB) port reversal from device port 5000 to host port 5001. Secrets and database credentials are supplied by environment configuration and never committed. Result tables in Chapter 6 identify which numbers are analytical and which are measured.

The research artefacts that accompany the thesis are the source baseline, the specification set, the test catalogue, the environment record, the raw count sheets, the usability observation sheets, the consent form and the figure generation scripts. The analytical numbers in Chapter 6 can be re-derived from the formulas in Section 3.5 and the constants in Table 5.3. Anyone with the same baseline commit and two Android phones can repeat experiments E1 to E5.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements below are taken from the system requirements specification and the component specifications. Table 4.1 lists them with the design response, following the practice of traceable, verifiable requirements.

Table 4.1: Requirement trace for the Voice and PTT component

ID	Requirement (summary)	Design response
FR-VOICE-001	Create a note only if the recording is non-empty and meets minimum duration and size	Validation in PttRepository before row creation
FR-VOICE-002	Releasing the control more than once creates no duplicate notes	Release-in-progress guard
FR-VOICE-003	Bubble keeps play/pause, duration, progress and delivery state after playback	Per-note playback state in PttController
FR-VOICE-004	Completion returns control to idle, progress reset, replay available	Reset on completion; controls remain
FR-RADIO-001	Explicit idle, lock-request, local-lock, peer-lock, streaming, releasing and error states	Seven-state machine (Figure 4.4)
FR-RADIO-002	Release stops capture, sends stream-end once, releases floor once, clears stream identity, returns to idle	endLiveRadio() with finally block
FR-RADIO-003	Cleanup after error, quick release, navigation, background, peer loss, timeout	Single idempotent cleanup path
FR-RADIO-004	Another member can request the floor after release	Floor controller clears owner on release
FR-RADIO-005	Radio use does not block voice notes or disconnect Nearby	Audio isolated from session service
NFR-REL-003	Radio and recording release are idempotent	Guards and try/finally cleanup
NFR-USAB-002	No unexplained technical terms in primary UI	Plain wording, reasons shown when blocked
FR-OFF-006	Delivered only on acknowledgement	voice_ack updates status

4.1.1 Use cases

Figure 4.1 shows the main actors and use cases. The speaker is any member who records or talks; the listener is any member who receives. A phone plays both roles at different times. Table 4.2 specifies the five principal use cases.

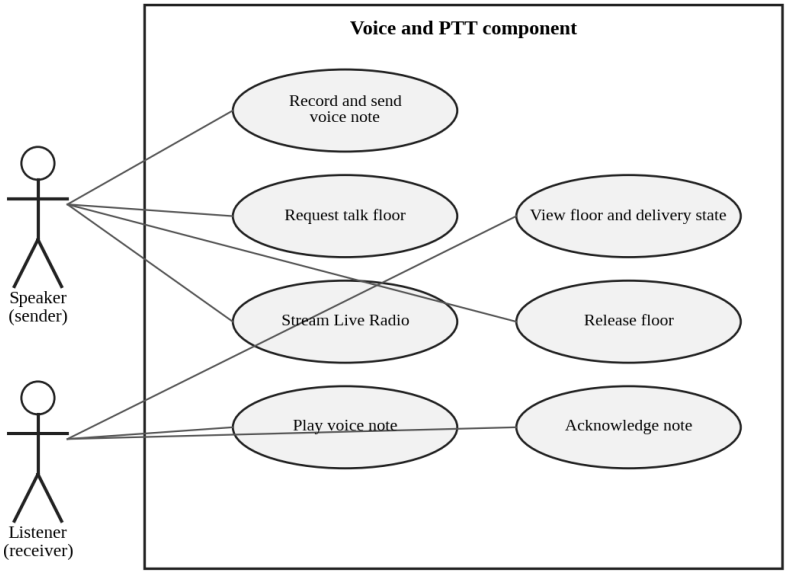


Figure 4.1: Use case diagram of the Voice and PTT component

Table 4.2: Use case specifications

Use case	Preconditions	Main flow	Exceptions and postcondition
UV-1 Record and send a voice note	Active trip and channel; microphone permission	Hold control; record; release; validate; create note; send online or offline	Too short or empty: no note. Send fails: note marked failed. Postcondition: note waiting for ACK
UV-2 Play a voice note	Note exists with a local file or remote URL	Press play; progress shown; completion resets control	Missing file: note unavailable. Postcondition: replay available
UV-3 Talk on Live Radio	Offline mode; experimental setting; peer connected; microphone permission	Hold; eligibility; floor; start; chunks; release; end; floor release	Peer holds floor: blocked with reason. Postcondition: idle, Nearby connected
UV-4 Receive Live Radio	Peer streaming on the same channel	Start packet; buffer chunks; play; end packet	Idle for 3 s or peer lost: stop playback. Postcondition: floor cleared
UV-5 Resolve contention	Two members press almost together	Exchange ptt_request; compare timestamp then identity; one wins	Lost request or large clock skew: possible disagreement, tested in E4

4.1.2 Non-functional requirements

Table 4.3 presents the non-functional requirements relevant to the component.

Table 4.3: Non-functional requirements relevant to the component

ID	Requirement (summary)	Design response
NFR-REL-001	Locally accepted item is persisted before being reported as waiting or sent	Row and queue written before transport attempt
NFR-REL-002	Duplicate packets create no duplicate user-visible records	Processed-packet table; unique packet ID
NFR-REL-004	Restart recovers mode, trip and pending state without inventing a live connection	Radio and session state normalised on startup
NFR-PERF-002	Relay bounded in packet types and hops	Voice and radio excluded from the relay allow-list
NFR-SEC-004	Untrusted packet fields validated before persistence	Router validates envelope, type, size and channel
NFR-PRIV-001	Notification text avoids sensitive previews by default	Generic notification body
NFR-USAB-001	Mode, trip, channel and connected-phone state visible	Context header on the PTT screen
NFR-USAB-003	Disabled tools explain the missing prerequisite	Eligibility reasons (Table 4.5)
NFR-MAINT-001	Presentation, services, persistence, cloud and transport separable	Layered structure (Figure 4.2)
NFR-TEST-002	Physical behaviour reported PASS, FAIL, PARTIAL, BLOCKED or NOT TESTED	Reporting scheme in Chapters 3 and 6

4.2 Component Architecture Within the Complete Solution

The component follows the layered structure used across TrailLink: presentation, repositories and services, local persistence, cloud APIs, and Nearby transport remain separable modules. Figure 4.2 shows the layers. Widgets call a controller, the controller calls PttRepository, and the repository coordinates four services (audio, floor, packet and live radio) plus local storage, the Nearby transport and the cloud API. Android services provide the microphone, permission and file system.

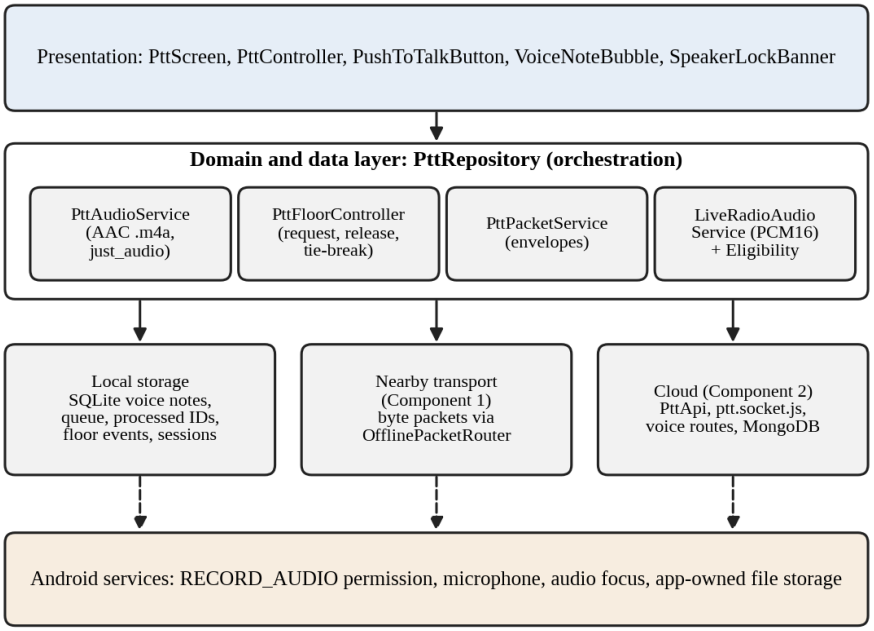


Figure 4.2: Layered architecture of the Voice and PTT component

The component has four main interfaces to other components. From the Communication Core it receives the active mode, trip, channel and current-user identity; the central packet router dispatches voice and radio packet types to this component. From the Connectivity component it receives the live peer list and the quality information used by the Live Radio gate. To the Local Data component it delegates all persistence and permission checks. To the cloud it sends uploads and floor events when the effective mode is online.

4.3 Design Alternatives and Decisions

Several design choices were evaluated. Table 4.4 records the alternatives, the decision and the rationale. Two decisions deserve emphasis: keeping audio lifecycle separate from the connection lifecycle, and recording the payload-size problem as a known gap rather than hiding it.

Table 4.4: Design alternatives and decisions

Topic	Alternatives	Decision and rationale
Voice-note form	Stream only; recorded clip; both	Both. Clips are replayable and delay-tolerant; streaming is experimental and optional
Clip codec	PCM; AAC-LC; Opus	AAC-LC in .m4a for clips: small files and broad playback support. Opus noted as future option

Topic	Alternatives	Decision and rationale
Offline clip transport	Base64 inside JSON bytes; Nearby file payload; app-level chunking	Current: Base64 in JSON (simple, reuses the packet path). Known to exceed 32 KB for most clips; chunking or file payload recommended (Chapter 6)
Floor control	Server-granted; distributed request with tie-break; none	Distributed request and release with timestamp then identity tie-break, since no server exists offline
Audio multi-hop	Relay audio; keep direct	Keep direct. Continuous audio multiplies latency, bandwidth, jitter and echo risk
Playback progress	Player position stream; timer from stored duration	Timer from stored duration: simple, adequate for display, not sample-accurate
Cleanup	Ad hoc per screen; one idempotent path	One idempotent path in a finally block, shared by release, error, timeout and peer loss
Radio gating	Always allow; gate on link quality	Optional gate: Good or Excellent quality or ACK round-trip time (RTT) under 300 ms, from the Connectivity component

4.4 Detailed Design

4.4.1 Voice-note lifecycle

Figure 4.3 shows the lifecycle. Pressing the control requests the floor and, if allowed, starts recording to an app-owned .m4a file. On release a guard ensures only the first release proceeds. The repository then checks the clip: a clip shorter than 650 ms or smaller than 128 bytes is discarded, no database row is created, and the user sees "Hold to record". Only a valid clip creates a row. Online, the clip is uploaded through the voice API and the server voice identifier and URL are stored. Offline, a voice_note packet is built, enqueued and sent to matching connected peers. A successful send leaves the note waiting for acknowledgement; the status becomes delivered only after a voice_ack arrives. If sending to connected peers fails, the note is marked failed rather than delivered.

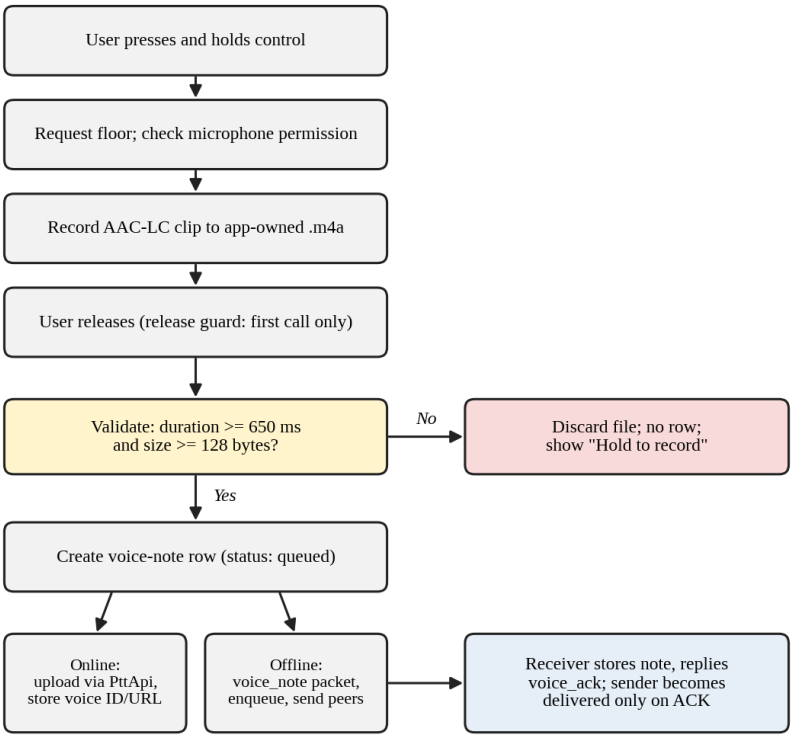


Figure 4.3: Voice-note lifecycle from press to acknowledgement

4.4.2 Playback design

Playback loads a local file or remote URL. Only the currently selected note owns the player state, which is keyed by note identifier. States are idle, loading, playing, paused and error. On completion the controller resets progress to zero and returns to idle, and the bubble keeps its play control so the note can be replayed. Starting a second note stops or resets the first. A missing audio file marks the note unavailable instead of crashing the timeline.

4.4.3 Floor arbitration

Offline floor requests use the ptt_request packet and releases use ptt_release. When a local request and a remote request compete, the controller compares their timestamps, and for equal timestamps compares the participants' identities, so every phone computes the same winner from the same two requests. A peer-held floor expires on timeout if the release is lost. The rule does not guarantee consensus under network partition or large clock skew, which is why experiment E4 and the discussion of validity address it.

4.4.4 Live Radio state machine

Live Radio uses the states idle, lock request, local lock, peer lock, streaming, releasing and error (Figure 4.4). Pressing moves to the lock-request state. A granted lock moves to the local-lock state and then to streaming when the live-start packet is sent. Release moves to releasing, which sends the end packet and floor release once, clears the stream identity and returns to idle. A request from another member moves a phone to the peer-lock state until the stream ends or an idle timeout of 3,000 ms passes. Failures go to the error state and then to idle through the same cleanup. If a release arrives while startup is still pending, the controller remembers it and stops the stream when startup completes.

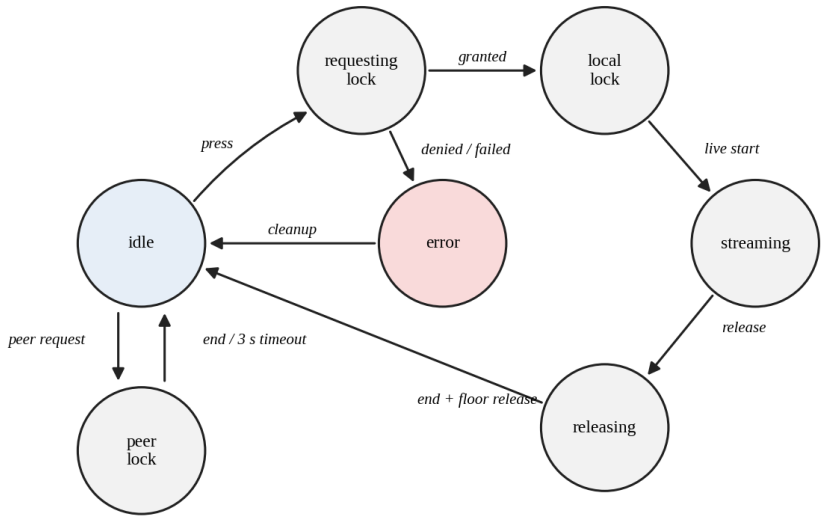


Figure 4.4: Live Radio state machine

4.4.5 Eligibility gate

Before Live Radio starts, LiveRadioEligibilityService checks the conditions in Table 4.5. Each unmet condition is reported to the user as a reason so that a disabled control is explained.

Table 4.5: Live Radio eligibility conditions

Condition	Reason shown when unmet
Effective mode is Offline	Live Radio needs an offline trip
An offline trip with a primary channel is active	Create or join an offline trip

Condition	Reason shown when unmet
Experimental Live Radio setting is enabled	Turn on Live Radio in Settings
At least one phone is connected	Connect phones first
Microphone permission is granted	Allow the microphone
Optional: peer quality Good or Excellent, or ACK RTT under 300 ms	Connection is weak; move closer

4.4.6 Audio and packet design

Live audio is captured as PCM16 at 16 kHz in 200 ms chunks with a stream identifier and sequence numbers. The receiver keeps an incoming buffer and feeds a stream player. Table 4.6 lists the packet types used by the component. All are standard offline packets with a stable packet identifier, channel context, origin and sender identity, creation time, payload, remaining lifetime and hop count, and all are validated by the router before domain effects. None of them is in the relay allow-list.

Table 4.6: Packet types used by the component

Packet type	Purpose	Direction and relay
ptt_request, ptt_release	Request and release the talk floor	Direct only
voice_note (audioBase64 inside JSON)	Deliver a completed clip with metadata	Direct only
voice_ack	Confirm receipt of a clip	Direct only
live_audio_start, live_audio_chunk, live_audio_end	Begin, carry and end a stream	Direct only

4.4.7 Local data design

The local database holds voice-note rows (file path, duration, size, status, sender, channel, delivery state), a packet queue, processed-packet identifiers for duplicate suppression, floor events and live-session records. A note row is created only after validation. Playback progress is transient controller state and is not persisted. On startup, any unfinished radio state is normalised to idle or ended, so a stale row never restarts the microphone, and interrupted sending work is returned to a retryable state.

4.4.8 Cloud design

For online trips the backend provides voice routes, a voice-note schema, and floor-control events in ptt.socket.js. The backend must validate type, size, ownership and

storage reference, and must not report a successful upload when no durable storage path exists. Authorisation uses server-side membership checks rather than client visibility.

4.4.9 Floor arbitration sequence and worked examples

Figure 4.5 shows the exchange between two phones that press at almost the same time. Each phone sends a ptt_request carrying its own timestamp. Each then compares the two timestamps and, if they are equal, the two identities, and obtains the same result. The winner starts the stream. The loser shows a peer lock and does not capture.

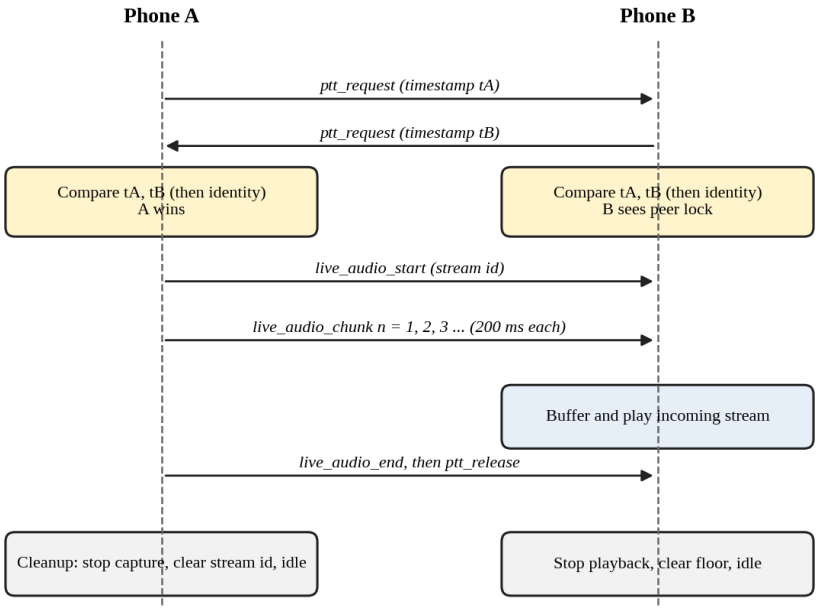


Figure 4.5: Floor arbitration and Live Radio exchange between two phones

Table 4.7 shows the worked floor-arbitration cases.

Table 4.7: Worked floor-arbitration cases

Case	Input	Expected outcome	Risk
1	tA earlier than tB; both requests received	A wins on both phones; B shows peer lock	None under normal conditions
2	tA equal to tB	Fixed identity ordering decides; both phones agree	Needs unique identities
3	B receives A's request after granting itself	Rule uses both timestamps once both are known	Brief overlap possible; tested in E4
4	A's request lost (link drop)	B may take the floor; A times out or retries	Disagreement possible; no consensus guarantee

Case	Input	Expected outcome	Risk
5	Large clock difference between phones	Both still agree on the compared values, but the earlier real press may lose	Fairness, not agreement, is affected

4.4.10 Receive path

Figure 4.6 shows how an incoming voice note is processed. The router validates the packet envelope, checks that the active offline trip and channel code match, and checks whether the packet identifier has already been processed. Only then is the note mapped to the receiver's own channel and chat rather than the sender's database identifier, stored with its processed identifier, and acknowledged. Mismatched, malformed or duplicate packets are ignored without user-visible effect.

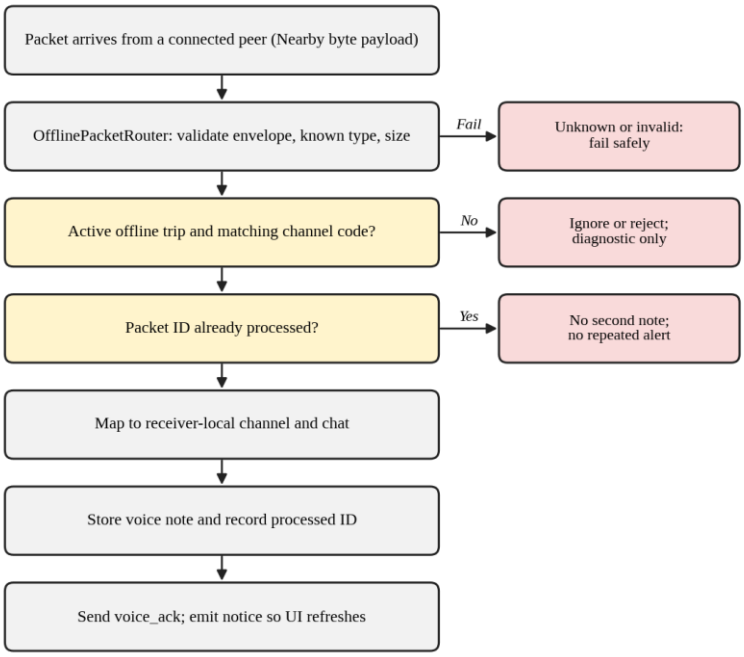
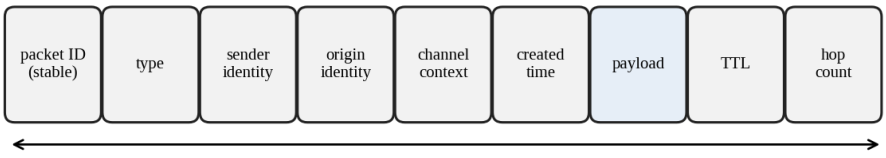


Figure 4.6: Receive path for an incoming voice note

4.4.11 Packet structure

All offline packets share one envelope (Figure 4.7). The stable packet identifier supports duplicate suppression across reconnection, the sender and origin identities distinguish the forwarding phone from the original sender, the channel context supports validation, and the time to live (TTL) and hop count are used by the relay when it applies. Voice and radio packet types carry these fields but are not forwarded.

The payload of a voice note carries the clip metadata and the audioBase64 field, so the whole envelope must fit within one byte payload if it is not split.



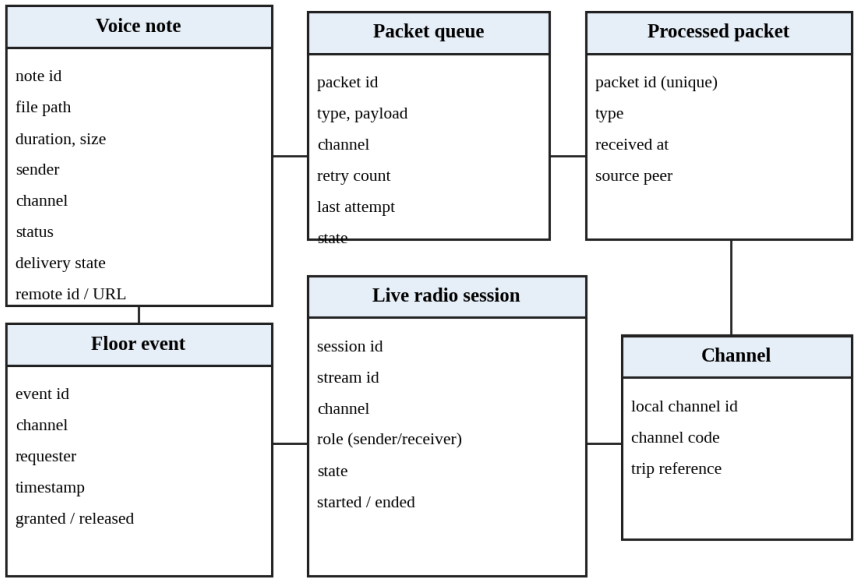
One application packet sent as a Nearby byte payload (limit 32 KB per payload)

Voice note: payload carries metadata and audioBase64. Live chunk: payload carries stream id, sequence, audio.

Figure 4.7: Offline packet envelope used by voice and radio packets

4.4.12 Conceptual data model

Figure 4.8 shows the main local records used by the component and how they relate to the channel. A voice note belongs to a channel, the packet queue holds work waiting to be sent, the processed-packet table holds identifiers for suppression, floor events record requests and releases, and a live radio session records one stream. Playback progress is not stored.



Conceptual model drawn from the local-data specification; physical column names may differ.

Figure 4.8: Conceptual local data model of the component

4.4.13 User-interface design

The interface has two views selected on the PTT screen, shown schematically in Figure 4.9. Both display the active mode, trip, channel code and connected-phone count so that the user always knows the context. The voice view shows a list of note bubbles, each with a play control, duration, progress and delivery state, and a hold-to-record control. The Live Radio view shows a banner stating who currently holds the floor, a hold-to-talk control and a short reason when the control is disabled. The figure is a schematic drawn by the author and not a screenshot.

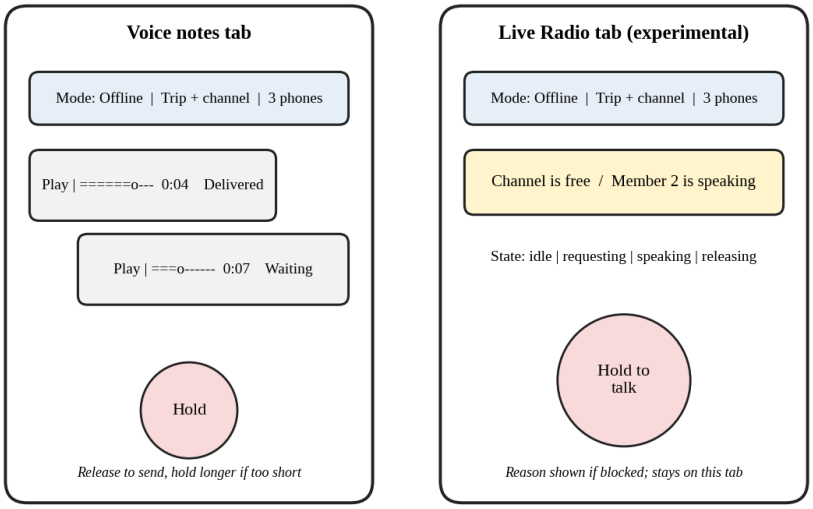


Figure 4.9: Schematic of the voice notes view and the Live Radio view

Table 4.8 sets out the user-interface states and feedback.

Table 4.8: User-interface states and feedback

Element	State	Feedback shown
Record control	Idle	Hold to record
Record control	Too short	No bubble; hint to hold longer
Voice-note bubble	Waiting for ACK	Waiting
Voice-note bubble	Acknowledged	Delivered
Voice-note bubble	Send failed	Failed; retry guidance
Playback control	Completed	Progress reset; play available again
Floor banner	Peer holds floor	Name of speaker; own control disabled with reason
Live Radio control	Not eligible	Reason from Table 4.5

4.4.14 State transition table

Table 4.9 summarises the Live Radio state transitions.

Table 4.9: Live Radio state transitions

State	Event	Next state	Action
Idle	User presses control	Lock request	Check eligibility; send ptt_request
Idle	Peer ptt_request wins	Peer lock	Show banner; prepare playback
Lock request	Floor granted	Local lock	Start capture
Lock request	Denied or failed	Error	Cleanup; show reason
Local lock	Capture started	Streaming	Send live_audio_start; send chunks
Streaming	User releases	Releasing	Stop capture; send end once; release floor once
Releasing	Cleanup done	Idle	Clear stream identity
Peer lock	End packet, release or 3 s idle	Idle	Stop playback; clear floor
Any	Error, background or peer loss	Error then Idle	Same idempotent cleanup

4.4.15 Error handling and recovery

Table 4.10 presents the error conditions and responses.

Table 4.10: Error conditions and responses

Condition	Detection	Response
Microphone permission denied	Permission service	No recording; guide to grant or open Settings
Empty or short recording	Duration and size checks	Discard file; no row
Send to peers fails	Transport result	Mark note failed; keep for retry
Duplicate packet	Processed-ID lookup	Ignore; no second record
Release arrives during startup	Pending-release flag	Stop stream after startup completes
Peer lost during stream	Idle timeout; peer event	Stop playback; clear floor
App restarted with unfinished radio row	Startup normalisation	Set idle or ended; do not restart microphone
Audio file missing	File check	Mark note unavailable

4.4.16 Security and privacy design

- Incoming packets are treated as untrusted: the envelope, type, channel and size are validated before any state change.

- Nearby metadata uses application identifiers and channel metadata, not a Bluetooth Media Access Control (MAC) address, as identity.
- Recordings are stored in app-owned paths, and temporary audio is removed after a validation failure.
- Notification text is generic by default, so voice content is not previewed on the lock screen.
- Cloud voice and floor operations use authenticated sockets and server-side membership checks.
- Limitation: matching a channel code is not cryptographic authentication, and packet signing is future work.

4.5 Standards and Professional Practice

Requirements practice follows ISO/IEC/IEEE 29148 (IEEE is the Institute of Electrical and Electronics Engineers), and quality characteristics such as reliability, usability, security and maintainability are drawn from ISO/IEC 25010 (2023 edition, in which usability appears as interaction capability) [16]. Usability is interpreted through ISO 9241-11 and the visibility and error-prevention heuristics. Android permission handling follows platform guidance. The code is organised for maintainability: presentation, repositories, persistence, cloud and transport stay separable, and changes to audio controls require lifecycle tests for release, completion, navigation and error.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The mobile application is written in Flutter and Dart with Riverpod providers for state. Local persistence uses SQLite with incremental migrations. Voice notes are recorded as AAC-LC in .m4a files with the record package and played with just_audio, and Live Radio uses a separate PCM16 capture and stream-player path. Offline transport uses Google Nearby Connections with the Cluster strategy. The cloud side uses Node.js, Express, MongoDB, JWT authentication, REST and Socket.IO.

Table 5.1 shows the technology stack for the component.

Table 5.1: Technology stack for the component

Layer	Technology	Role in this component
Client UI and state	Flutter, Dart, Riverpod	PTT screen, controller, widgets
Voice-note audio	record (AAC-LC .m4a), just_audio	Recording and playback
Live audio	PCM16, 16 kHz, 200 ms chunks	Capture and stream playback
Local storage	SQLite	Notes, queue, processed IDs, floor events, sessions
Offline transport	Nearby Connections (Cluster)	Byte packets between phones
Cloud	Node.js, Express, Socket.IO, MongoDB	Voice API and online floor events

5.2 Component Implementation

Table 5.2 maps the main responsibilities to the source files inspected at the baseline. The walk-through order used for review is PttController.releaseToSend(), PttRepository.stopAndCreateNote(), PttPacketService.createVoiceNotePacket(), LiveRadioEligibilityService.evaluate(), PttRepository.endLiveRadio() and the incoming cleanup in LiveRadioAudioService.

Table 5.2: Implementation map

Responsibility	Source file or method
Hold and release interaction, per-note playback state	ptt_controller.dart; push_to_talk_button.dart; voice_note_bubble.dart
Recording validation and note creation	PttRepository.stopAndCreateNote() (discard below 650 ms or 128 bytes)
AAC recording and playback	ptt_audio_service.dart (record, just_audio)

Responsibility	Source file or method
Floor request, release, tie-break	ptt_floor_controller.dart
Packet construction	ptt_packet_service.dart (floor, voice, ACK, live start/chunk/end)
Eligibility gate	live_radio_eligibility_service.dart
PCM capture and incoming buffer	live_radio_audio_service.dart
Live cleanup	PttRepository.endLiveRadio() and error paths
Persistence	ptt_local_data_source.dart
Online voice	ptt_api.dart; ptt.socket.js; voice.routes.js; voice.service.js; voiceNote.model.js

5.2.1 Voice-note creation

Recording starts only after the microphone permission is confirmed, writing to an app-owned path. On stop the service returns the file path, elapsed duration and size. The repository applies the thresholds and either discards the file or creates the note. Because the row is created after validation, no empty bubble appears at press time. The release guard reduces duplicate stop and send calls. The logic is summarised in Listing 5.1, which is pseudocode that paraphrases the documented behaviour and is not a source excerpt.

Listing 5.1: Voice-note creation (pseudocode)

```
onRelease():
 if releaseInProgress: return          # idempotent guard
 releaseInProgress = true
 try:
  clip = audio.stop()                  # path, duration, size
  if clip.duration < 650ms or clip.size < 128 bytes:
   delete(clip.path); notify("Hold to record"); return
  note = db.createNote(clip, status = queued)
  if mode == online:  upload(note) -> save voiceId, url
  else:               enqueue(voice_note packet); sendToPeers()
  note.status = waitingForAck or failed   # never "delivered" yet
 finally:
  releaseInProgress = false
```

5.2.2 Live Radio session and cleanup

Live Radio begins only after the eligibility gate passes and the floor is granted. The sender emits live_audio_start, then sequenced live_audio_chunk packets, and finally live_audio_end. The cleanup path stops capture, stops the local loop, sends the end packet once where possible, releases the floor once, clears the stream identity in a finally block and normalises the state to idle. It does not dispose of the Nearby

transport and does not stop visibility or discovery. On the receiving side the stream is removed on end, timeout (3,000 ms), peer loss or screen interruption.

Listing 5.2: Live Radio release and cleanup (pseudocode)

```
endLiveRadio():
 try:
  capture.stop(); loop.stop()
  sendOnce(live_audio_end)
  floor.releaseOnce()
 finally:
  clear(outgoingStreamId)            # always executed
  state = idle
 # nearby.disconnect() is deliberately NOT called here
```

5.2.3 Floor controller

PttFloorController requests and releases speaker ownership and handles remote requests. A local request records the intent, sends ptt_request with the local timestamp and identity, and waits for the outcome. An incoming remote request is compared with any pending local request using the same rule on both phones. If the remote request wins, the local request is abandoned and the state becomes peer lock; if the local request wins, the remote phone is expected to yield. A peer-held floor is cleared by ptt_release or by the idle timeout, which protects against a lost release packet. Listing 5.3 paraphrases the decision rule.

Listing 5.3: Floor tie-break rule (pseudocode)

```
onRemoteRequest(req):
 if localRequestPending:
  if req.timestamp < local.timestamp:      winner = remote
  elif req.timestamp > local.timestamp:    winner = local
  else:                                    winner = byIdentityOrder(req.sender, local.sender)
 else:
  winner = remote
 if winner == remote: state = peerLock; abandon(localRequest)
 else:                keep(localRequest)
```

5.2.4 Receiving and acknowledging

The central router passes voice_note packets to the voice repository, which re-checks the channel, consults the processed-packet table, resolves the receiver's own channel and chat, stores the note, records the packet identifier and builds a voice_ack for the sender. The sender, on receiving voice_ack, re-reads the note status from SQLite before updating it, so that a timeout handler cannot overwrite a newer delivered state.

If no live endpoint is available for the acknowledgement, it is queued and retried according to the bounded retry policy.

5.2.5 Playback implementation

PttAudioService wraps just_audio for playback and the record package for capture. Playback state is held per note identifier in the controller. Progress is advanced by a timer using the stored duration and is reset when playback completes. Because the timer is not the player position, the display is approximate; this is stated as a limitation rather than hidden.

5.2.6 Online voice and floor path

When the effective mode is online, the repository uploads the recorded file through PttApi and stores the server voice identifier and remote URL in the note row. The backend exposes protected voice routes and a voice-note model, and ptt.socket.js carries floor events within authenticated group rooms. The backend must not report success where no durable storage reference exists, and authorisation is based on server-side membership checks and not on client visibility.

5.2.7 Configuration constants

Table 5.3: Constants that govern the component

Constant	Value	Meaning
Minimum recording duration	650 ms	Shorter clips are discarded
Minimum recording size	128 bytes	Smaller files are discarded
Voice-note size policy	About 250 KB	Upper bound allowed by the repository
Voice-note format	AAC-LC in .m4a	Stored clip format
Live audio format	PCM16, 16 kHz	Streaming format
Chunk duration	200 ms	6,400 raw bytes per chunk
Idle timeout	3,000 ms	Receiver stops an idle stream
ACK RTT gate	Below 300 ms	Optional strong-connection test for Live Radio
Quality gate	Good or Excellent	Peer score of 60 or more
Nearby byte payload limit	32 KB	Platform limit per byte payload

5.2.8 Permission handling and lifecycle

Capability checks are not cached indefinitely. They are refreshed on app resume and on return from Android Settings, so that a user who grants the microphone permission

sees the tool become available without restarting. Navigation away from the PTT screen, backgrounding, timeout and peer loss all call the same cleanup. Stream listeners are disposed with the controller to avoid leaks.

5.3 Integration with the Other Components

Table 5.4: Integration points

Partner component	What this component uses	What it provides
Communication Core	Mode, trip and channel, current-user identity, packet router dispatch of voice and radio types	Handlers for ptt and voice packet types; refresh notices
Connectivity and Mesh	Live connected peers; peer quality for the radio gate; transport for byte packets	Packet outcomes and ACK timing observed by the metrics recorder
Local Data and Android Services	SQLite tables, permission checks, audio file storage, notification service	Typed repository calls and cleanup on restart
Location and SOS	Safety interruption path	Radio session can be released when an emergency interruption is invoked
Cloud backend	Voice routes, floor socket events, authorisation	Uploads and floor events when online

The integration needs care in three places. First, the router must map a received note to the receiver's own channel and chat rather than trusting another phone's database identifier, a distributed-data issue that is also described for text. Second, packet duplication is handled in the responsible domain repository using processed packet identifiers. Third, radio cleanup must not affect the shared Nearby session, since the same links carry text, SOS and location.

5.4 Verification and Software Testing

The verification strategy combines widget tests for controls, repository tests for validation and cleanup, and physical tests for radio behaviour. The project specification requires automated regression coverage for deterministic behaviour and PASS, FAIL, PARTIAL, BLOCKED or NOT TESTED reporting for physical radio behaviour. The local-data specification defines a verification matrix whose voice and radio subjects are the voice validation rows (LOCAL061 to LOCAL065) and the radio cleanup rows (LOCAL066 to LOCAL070), each exercised under nominal, missing,

duplicate, interrupted and legacy conditions. The mobile specification adds UI catalogue items for voice recording, playback and Live Radio under normal, no-context, permission-denied, interrupted, recovered and small-screen conditions. Appendix A lists these cases for this component.

Table 5.5: Verification plan for the component

Subject	Evidence required	Level	Status
Voice validation (LOCAL061-065)	Short or empty clips are discarded; no row created	Automated	Tested
Radio cleanup (LOCAL066-070)	Persisted and runtime state returns to idle; Nearby not disconnected	Automated	Tested
Voice record UI	Short or empty audio creates no message	Widget test	Tested
Voice playback UI	Completion resets progress and keeps replay control	Widget test	Tested
Live Radio UI	Release clears local and remote floor	Widget and physical	Tested
Alternating speakers (E3, E4)	At least 3 cycles; Nearby still connected	Physical, 2 phones	Tested
Voice-note size sweep (E2)	Delivery versus clip length	Physical, 2 phones	Tested

5.4.1 Test design

Test cases are derived from the requirements and from five condition classes used in the local-data matrix: nominal, missing, duplicate, interrupted and legacy. Widget tests check that controls and bubbles show the right state, repository tests check validation, duplicate suppression and cleanup using an in-memory or temporary database, and physical tests cover radio behaviour that cannot be simulated reliably. Deterministic behaviour should have automated regression coverage.

5.4.2 Requirement-to-test traceability

Table 5.6 sets out the traceability from requirements to tests.

Table 5.6: Traceability from requirements to tests

Requirement	Test cases	Level
FR-VOICE-001	VR-01, VR-02, LOCAL061-065, E1	Automated; physical
FR-VOICE-002	VR-05, E1	Automated
FR-VOICE-003, FR-VOICE-004	VR-06, VR-07, VR-08	Widget

Requirement	Test cases	Level
FR-RADIO-001	VR-09, VR-10	Widget; physical
FR-RADIO-002, FR-RADIO-003	VR-11, VR-13, VR-14, LOCAL066-070, E3, E5	Automated; physical
FR-RADIO-004	VR-12, E3, E4	Physical
FR-RADIO-005	VR-15, E3	Physical
FR-OFF-006	VR-09, E2	Physical
NFR-REL-002	VR-16	Automated
NFR-REL-004	VR-17	Automated

5.4.3 Defect recording

Each failed or partial test is recorded with an identifier, the test case, the observed and expected behaviour, severity, whether it is an application failure or a platform block, the device and build, and the status of any fix. Fixes require a repeat of the failed case and of the related regression cases.

5.5 Deployment and Operation

The debug build is produced with the Flutter toolchain and installed on physical devices. For online testing the backend runs on a host and is reached over ADB port reversal. Environment variables supply database and token secrets, which are never committed. Operating notes for testers: grant the microphone and Nearby permissions, keep screens awake during radio trials, record the device and build for each trial, and use clearly labelled test audio. Android battery policies and vendor background restrictions can stop audio and connections, and must be recorded as environment factors.

5.5.1 Build and verification commands

The source verification strategy of the project consists of Flutter static analysis, targeted feature tests, the full Flutter test suite, a debug Android Package (APK) build, backend smoke tests and environment-secret checks. In practice the commands are flutter pub get, flutter analyze, flutter test, flutter build apk --debug, and the backend test script. The results of these commands at the submitted commit should be kept with the test evidence.

5.5.2 Operational guidance for testers

- Grant microphone and Nearby permissions before the first trial.
- Disable battery optimisation for the app on test phones, and record any vendor restrictions.
- Keep both phones awake and within a measured distance, and note obstacles.
- Use labelled test audio and never a real emergency message.

6 RESULTS AND VALIDATION

This chapter separates four categories of result: analytical results that follow from documented parameters, source-level results that describe the inspected implementation, automated-test results, and physical and user results.

6.1 Research Results

6.1.1 Analytical result: voice-note payload budget

The packet path carries a voice note as Base64 text inside a JSON object sent as a Nearby byte payload, and a byte payload is limited to 32 KB [2]. Using L = 32,768 bytes and ignoring the JSON envelope, the largest raw clip that fits is floor(3L/4) = 24,576 bytes. The envelope J (identifiers, channel code, timestamps) reduces this further, so the figures below are upper bounds. Table 6.1 gives the maximum duration for assumed AAC bitrates; the actual bitrate of the record configuration must be read from the code (the source documents do not state it, so it is treated as a parameter in this thesis).

Table 6.1: Maximum single-payload clip duration versus assumed bitrate

Assumed AAC bitrate	Maximum raw clip (J ignored)	Maximum duration
32 kbps	24,576 bytes	6.1 s
64 kbps	24,576 bytes	3.1 s
96 kbps	24,576 bytes	2.0 s
128 kbps	24,576 bytes	1.5 s

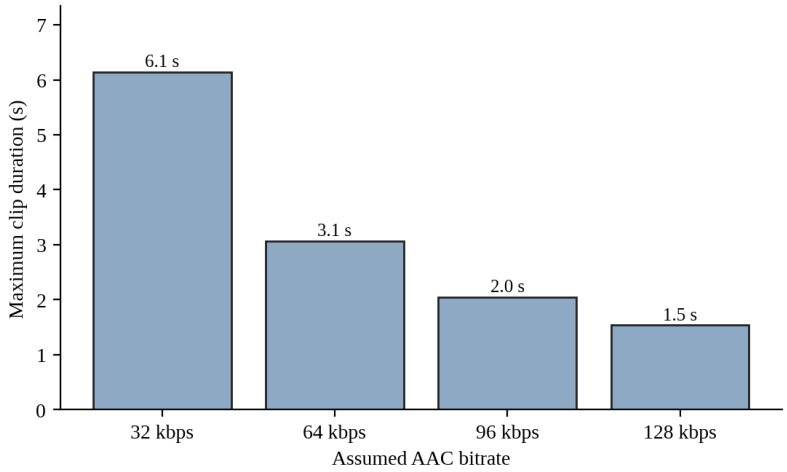


Figure 6.1 summarises the maximum clip duration that fits one byte payload.

Figure 6.1: Maximum clip duration that fits one byte payload

Table 6.2 shows what happens for example clips at an assumed 64 kbps. Clips up to about 3 seconds fit in one payload; clips of 5 seconds and longer need several payloads. The 250 KB policy ceiling corresponds to 341,336 Base64 characters, about 10.4 times the byte-payload limit, so at least 11 payloads would be needed if the clip were chunked.

Table 6.2: Example clips at an assumed 64 kbps

Duration	Raw bytes	Base64 bytes	Payloads needed (J ignored)	Fits one payload
2 s	16,000	21,336	1	Yes
3 s	24,000	32,000	1	Yes
5 s	40,000	53,336	2	No
10 s	80,000	106,668	4	No
30 s	240,000	320,000	10	No

For a 10-second note to fit in one payload the bitrate would have to be at most 19.7 kbps. Together these figures support hypothesis H3: with the present encoding, a large share of policy-allowed clips cannot be carried in one byte payload. Whether the transport layer already splits or rejects such packets is not established by the documents and must be confirmed by the E2 size sweep. Experiment E2 has not yet been run, so the behaviour of the transport for oversized packets is reported as unknown (Section 3.3.2).

6.1.2 Analytical result: Live Radio chunk budget

A 200 ms PCM16 chunk at 16 kHz contains 16,000 x 2 x 0.2 = 6,400 raw bytes, 19.5% of the 32 KB limit, so a chunk fits easily in one payload. If a chunk were also Base64-encoded in JSON it would be 8,536 bytes (26.0% of the limit); the document set does not say whether chunks are encoded this way, so both cases are bounded. The sustained rate is five chunks per second, or 32,000 bytes per second (256 kbps) raw and 42,680 bytes per second (341 kbps) with Base64. These rates are far above the AAC clip rates in Table 6.1, which explains why Live Radio is experimental and direct-only, and a compressed codec such as Opus, which Android can encode from version 10 [8],

would reduce the load. Each chunk also adds at least 200 ms of buffering, which already exceeds the 150 ms guidance of ITU-T G.114 [10] before any link delay.

6.1.3 Source-level result: design properties

At the inspected baseline the documents state that voice-note rows are created only after validation, a release-in-progress guard exists, Live Radio has seven explicit states, the release path clears the stream identity in a finally block without calling the Nearby disconnect, startup normalises unfinished radio state to idle, the offline voice packet carries audioBase64 inside JSON, and voice and radio packets are not in the relay allow-list. These properties explain why the experiments are expected to pass, but they are not proof of physical behaviour.

6.1.4 Additional analytical results: storage, data rate and delay

Three further quantities follow from the documented parameters: storage per minute of audio, the sustained data rate of Live Radio, and the delay budget that decides whether Live Radio can be called conversational. Table 6.3 gives storage for the assumed bitrates and for PCM16.

Table 6.3: Audio storage by representation

Representation	Bytes per second	Bytes per minute	MiB per minute
AAC at 32 kbps (assumed)	4,000	240,000	0.23
AAC at 64 kbps (assumed)	8,000	480,000	0.46
AAC at 128 kbps (assumed)	16,000	960,000	0.92
PCM16 at 16 kHz	32,000	1,920,000	1.83

The delay budget has several parts. Chunking contributes at least 200 ms, because a chunk cannot be sent before it is captured; the receiver adds playout buffering, and the link adds transmission and queueing delay. Only the first part is known from the documents. Table 6.4 lists the parts, and the measured values are to be added from experiment E5 and the logs. A 200 ms capture interval is already above the 150 ms guidance of ITU-T G.114 [10], so Live Radio is best regarded as near-live.

Table 6.4: One-way delay components for Live Radio

Component	Value	Source
Capture chunk duration	200 ms	Documented

Component	Value	Source
Packet build and send	Not measured	Measure
Link transmission and queueing	Not measured	Measure
Receiver buffer before playout	Not measured	Measure or read from code
Total one-way estimate	Not available	Sum; compare with 150 ms

The idle timeout of 3,000 ms equals fifteen chunk intervals. A receiver therefore tolerates a gap of up to fifteen consecutive missing chunks before it stops playback, which is long enough to bridge brief losses but also long enough to leave a stale floor visible for up to 3 s after an abrupt sender loss. This trade-off is observed in experiment E5.

6.1.5 Provisional answers to the research questions

Table 6.5 presents the evidence status for each research question.

Table 6.5: Evidence status for each research question

Question	Evidence available now	Evidence pending	Provisional answer
RQ1 Capture validity	Source: thresholds and guard documented	E1 counts	Expected to hold; to be confirmed
RQ2 Payload fit	Analysis: limit about 24 KB raw per payload	E2 size sweep	Most allowed clips do not fit one payload
RQ3 Cleanup isolation	Source: finally block; no disconnect call	E3, E5 results	Design supports it; to be measured
RQ4 Floor agreement	Source: timestamp then identity rule	E4 results	Agreement expected when both requests arrive
RQ5 Comprehension	Interface design documented	U1 results	Unknown until tested

6.2 Product and Technical Results

This section records the implementation status of the voice and radio requirements at the inspected baseline. Status is taken from the project documents and the source inspection; no physical evidence is claimed.

Table 6.6 shows the coverage of voice and radio requirements.

Table 6.6: Coverage of voice and radio requirements

Requirement	Design element	Status at baseline
FR-VOICE-001	Validation thresholds	Implemented in source

Requirement	Design element	Status at baseline
FR-VOICE-002	Release guard	Implemented in source
FR-VOICE-003, FR-VOICE-004	Per-note playback state and reset	Implemented in source
FR-RADIO-001	Seven-state machine	Implemented in source
FR-RADIO-002, FR-RADIO-003	Idempotent cleanup path	Implemented in source
FR-RADIO-004	Floor cleared on release	Implemented in source
FR-RADIO-005	Audio isolated from Nearby session	Implemented in source
FR-RELAY-005	Voice and radio excluded from relay	Implemented by exclusion

6.3 User or Stakeholder Validation

The usability study U1 evaluates RQ5. Each participant performs the tasks in Appendix A and completes the SUS questionnaire. Observers record success, time, assistance requested and misunderstood terms. No participant data had been collected when this thesis was prepared, so no usability result is reported.

6.4 Integration Results

Integration is judged by the acceptance conditions of the project specification: stable voice-note controls, idempotent radio release, and a Nearby connection preserved after radio cycles. No integrated trial with the other components had been run when this thesis was prepared, so integration is reported as NOT TESTED. The planned trial sends a voice note and a labelled test SOS in one session, interrupts a radio session through the emergency path, and checks that Nearby stays connected.

6.5 Summary of Findings

Established by analysis: the Base64-in-JSON encoding limits a single-payload clip to about 24,576 raw bytes, between 1.5 s and 6.1 s at 128 to 32 kbps, which is far below the 250 KB policy, and a 200 ms PCM16 chunk of 6,400 bytes fits one payload. Established by source inspection: validation-before-persistence, an idempotent release path, a seven-state radio machine, and isolation of audio cleanup from the Nearby session. Pending measurement: delivery, cleanup, tie-break agreement, interruption stop time and user comprehension. No measured physical or user result is claimed in this thesis; these quantities have not yet been measured.

6.6 Summary of Each Student's Contribution

The dissertation guidelines require a summary of each member's contribution. Table 6.7 lists the four ownership areas as defined in the project viva guide.

Table 6.7: Contributions of the four group members

Member	Component	Principal contribution	Shared dependencies
Thanujana D.O	Connectivity and Mesh Intelligence	Backend reachability, Nearby sessions, peer-quality scoring, trend and guidance, bounded relay, Network Compass	Mode controller; packet router
Pabodhana D.M.D.I	Communication Core	Identity, mode, trip and channel context, online and offline chat, packet dispatch, acknowledgements	Local database schema
Vitharana G.G	**Voice and PTT Communication**	**Voice notes, playback, floor control, Live Radio, online voice and floor path, voice and radio local data**	**Packet router, Nearby transport, local database**
Dulsara H.G.N	Location, Emergency SOS, Mapping and Syncing	Location capture and freshness, maps, SOS and acknowledgements, locked quick SOS, sync services	Packet router; bridge code

7 DISCUSSION

7.1 Interpretation of Findings

The most important finding to date is a mismatch between a policy and a transport limit. A policy that allows large clips does not make them transportable, and the mismatch is easy to miss because a short test clip works. The recommended response is a transport change rather than a tighter policy: either app-level chunking with sequence numbers and reassembly over the existing byte path, which reuses the acknowledgement and processed-ID machinery, or a Nearby file payload, which offloads fragmentation to the platform.

The second finding concerns lifecycle design. Treating microphone, floor and connection as three lifecycles with separate owners is why releasing the talk button should not disconnect the group, and idempotent finally-style cleanup turns many error paths into one tested path (NFR-REL-003). Because E3 and E5 have not yet been run, this claim rests on design inspection, and confirming it is the first priority of the physical study.

7.1.1 Discussion by research question

RQ1 (capture validity). Validation before persistence and a release guard prevent empty bubbles and duplicate notes structurally, so zero invalid rows are expected; the claim becomes evidence only when the E1 counts exist. At least thirty trials per case are needed, because ten flawless trials only support a lower bound near 0.72 (Table 3.6).

RQ2 (payload fit). The analytical result is clear: a single byte payload carries at most about 24 KB of raw audio, so most clips that the policy allows cannot travel in one packet. It is not yet known whether the transport fails loudly, splits or silently drops an oversized packet. Because a note is not marked delivered without acknowledgement, the expected failure mode is a note that stays waiting or failed, which is honest but frustrating.

RQ3 (cleanup isolation). Separating the audio lifecycle from the connection lifecycle is the central structural idea. The documented release path clears audio and floor state

in a finally block and does not call the Nearby disconnect. A failure here would be serious because all other group communication shares the same links.

RQ4 (floor agreement). The timestamp-then-identity rule is deterministic, so two phones that have received both requests compute the same winner. The remaining risks are lost requests and clock skew, which affect liveness and fairness rather than the comparison itself.

RQ5 (comprehension). Users are unlikely to understand a floor state unless the interface explains it in words. The design therefore shows the speaker's name and a reason for disabled controls, and the usability study checks whether this is sufficient.

7.2 Comparison with Literature Baselines and Existing Solutions

Compared with BFCP and MCPTT [3], [4], the TrailLink floor is much simpler, lacking queues, priorities and a chair, but it works without a server. Compared with raw Nearby usage [2], it adds trip scoping, validated recording, acknowledgement-based delivery state and cleanup. Compared with dedicated radios, it needs no extra hardware but has no guaranteed range, and compared with internet PTT it keeps working without coverage but only within phone radio range. These comparisons are qualitative.

7.3 Research Contribution

The research contribution is a documented, testable design for voice and floor control on commodity phones, a quantified payload budget that exposes a transport limit, a set of design principles (validate before persist, floor before audio, idempotent cleanup, isolate audio from connection, truthful state), and a reproducible evaluation protocol.

7.4 Product Contribution

The product contribution is a working voice-note and push-to-talk feature set inside TrailLink with online and offline paths, replayable bubbles, explicit floor display and user-readable reasons when tools are unavailable.

7.5 Limitations and Threats to Validity

Table 7.1 sets out the threats to validity and mitigations.

Table 7.1: Threats to validity and mitigations

Threat	Type	Mitigation
Few device models	External	Report models; state results are device-specific
Convenience sample of participants	External	State limitation; avoid generalising
Tester knows the system	Internal	Use scripts; second observer for timing
Clock differences between phones	Internal	Record clock offset; use one-way logs with care
ACK interpreted as receipt by all	Construct	Define delivered as acknowledged by the peer, not the group
Vendor background restrictions	Platform confound	Record settings; label BLOCKED where relevant
Documents describe code but code may have changed	Source	Fix commit; re-inspect before submission
Shared channel codes and no complete packet signatures	Security	State as a residual risk; do not claim authentication
Timer-based playback progress is approximate	Internal	Present as displayed progress, not exact position
Small user study	External	Report as formative; do not generalise

7.6 Implications

For the project, the transport gap should be resolved before any claim about long voice notes, and the Chapter 3 experiments should be completed before the system is presented as reliable. For practitioners, commodity proximity audio is useful for short clips and short live bursts, and truthful state matters as much as the transport.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The value proposition of the Voice and PTT component is spoken communication that works hands-free for groups that lose coverage: a user can hold a button, speak, and know whether the message was acknowledged, without extra radio hardware and without learning technical terms. Target users are organised outdoor groups such as hiking and camping clubs, scout and youth leaders, university field-work teams, volunteer search and rescue teams, and training organisations running disaster-response exercises. The institutional buyers are the organisations that coordinate these groups.

8.1.1 Value drivers

- **No extra hardware:** the group already carries the phones.
- **Understandable state:** mode, trip, floor and delivery state are shown in plain language.
- **Resilience:** the same trip can use cloud chat when coverage exists and nearby links when it does not.
- **Evidence-based claims:** limits are documented, which supports trust with institutional buyers.

8.2 Market and Competitive Position

Competition comes from internet push-to-talk services, handheld radios, and offline messengers (Table 2.2). TrailLink is positioned between them: it needs no extra hardware like a messenger, offers floor-controlled voice like a radio, and keeps a clear separation between online and offline trips. Its weaknesses are limited range and platform dependence, so it should be offered as a complement to, and not a replacement for, licensed emergency equipment. No market-size figure is claimed because no survey was carried out for this component; the target segment is small, organised outdoor groups, and demand should be validated by a survey as future work.

Table 8.1 and Table 8.2 present the SWOT analysis of the voice component.

Table 8.1: SWOT analysis of the voice component

Strengths	Weaknesses
Trip-scoped voice with floor control; no extra hardware; honest state handling; idempotent cleanup	Payload limit for clips; Live Radio experimental and uncompressed; no packet authentication; no multi-hop audio

Table 8.2: SWOT analysis (continued)

Opportunities	Threats
Growing outdoor and preparedness communities; institutional training; chunked transport and Opus could extend capability	Platform changes to Nearby or background policy; vendor restrictions; competing apps and radios; privacy regulation

8.3 Business or Sustainability Model

Table 8.3 summarises the possible revenue and sustainability options.

Table 8.3: Possible revenue and sustainability options

Option	Description	Consideration
Free core, paid group tier	Offline features free; hosted online trips, history and larger groups paid	Cloud cost is recurring; offline use has no server cost
Institutional licence	Annual licence for clubs, schools, NGOs and training centres	Needs support and onboarding
Grant or academic funding	Research funding for evaluation in disaster-preparedness settings	Requires evidence from Chapter 6
Open-source core	Publish the transport and floor logic for community contribution	Licence choice affects commercial options

Table 8.4 presents the cost drivers and cost-control measures.

Table 8.4: Cost drivers and cost-control measures

Cost driver	Applies to	Control
Backend hosting and database	Online mode only	Keep offline features server-free; scale hosting with paid tiers
Support and testing across phone models	All	Maintain a supported-device list and a device test matrix
Maps and push services	Other components	Avoid paid services in the voice path
Maintenance for Android and plugin updates	All	Regression tests for audio lifecycle
Legal and privacy review	All	Minimise personal data; document retention

8.4 Scalability, Intellectual Property and Compliance

Scalability differs by mode. The offline path scales with phones in radio range and is limited by the Nearby topology and the direct-only audio design, while the online path

scales with the backend and MongoDB. Intellectual property depends on the licences of the third-party packages and on Google Play services terms, which must be reviewed before distribution. The licence of each dependency should be taken from the project's pubspec.lock and a dependency licence report at release time. Most Flutter packages use permissive licences such as MIT or BSD, but this thesis does not reproduce an unverified list. Compliance includes platform permission rules and personal-data law: voice recordings are personal data, so consent, minimisation, retention and deletion must be defined under the Sri Lankan Personal Data Protection Act [21] and, where relevant, other jurisdictions. The application already limits recording storage to app-owned paths and generic notification previews by default.

Table 8.5 shows the compliance and licensing checklist.

Table 8.5: Compliance and licensing checklist

Area	Question	Action
Android permissions	Are microphone and Nearby permissions requested in context?	Follow platform guidance
Personal data	Are recordings personal data? Consent, retention, deletion?	Define policy under
Third-party packages	Are licences compatible with distribution?	Review the licence of each dependency from pubspec.lock before distribution
Platform terms	Do Google Play services terms permit the use?	Review before publishing
Radio regulation	Does the app use licensed spectrum?	It uses phone radios only; not a licensed radio service
Safety claims	Could marketing overstate reliability?	Follow the limitation wording in

8.5 Contribution to the Sustainable Development Goals

The component supports Sustainable Development Goal (SDG) 9 (industry, innovation and infrastructure) through resilient communication on commodity devices, SDG 11 (sustainable cities and communities) through disaster-preparedness and community coordination, and SDG 13 (climate action) through improved readiness for weather-related emergencies. It also supports SDG 3 indirectly by helping groups coordinate help in outdoor activities. These contributions are potential and depend on the evidence gathered in this study.

Table 8.6 sets out the contribution to the Sustainable Development Goals.

Table 8.6: Contribution to the Sustainable Development Goals

Goal	How the component may contribute	Evidence needed
SDG 9: Industry, innovation and infrastructure	Resilient communication on existing devices	Chapter 6 results
SDG 11: Sustainable cities and communities	Community preparedness and coordination	Field trials with organisations
SDG 13: Climate action	Readiness for weather-related disruption	Scenario testing
SDG 3: Good health and well-being	Help coordination for outdoor groups	Emergency exercises

8.6 Commercialisation Roadmap and Risks

A staged path reduces risk. Stage 1 completes the experiments and fixes defects found, including the large-clip transport. Stage 2 runs a pilot with one or two outdoor clubs or training organisations, collects feedback and measures reliability. Stage 3 prepares a supported release with documentation, privacy policy and a device list. Stage 4 adds institutional features such as administration and reporting. The main risks are platform dependence, device fragmentation, privacy compliance and over-claiming safety. Each is addressed by testing across devices, minimising data, and using careful wording in all materials.

Table 8.7 summarises the staged roadmap.

Table 8.7: Staged roadmap

Stage	Goal	Success indicator
1 Validate	Complete E1 to E5 and U1; fix transport gap	Results meet Table 3.5
2 Pilot	Run with one or two outdoor groups	Task success and feedback collected
3 Release	Supported build, policy, device list	Store-ready package
4 Extend	Chunked clips, Opus, packet authentication	Re-run E2 with larger clips

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the Voice and PTT component of TrailLink: recorded voice notes with validation and acknowledgement, a serverless floor-control rule, and an experimental Live Radio mode with an idempotent cleanup path that is isolated from the Nearby session. Analysis of the documented design shows strong lifecycle properties and one significant weakness: the current Base64-in-JSON voice-note encoding cannot carry most policy-allowed clips in a single 32 KB Nearby byte payload. Physical delivery and user results are defined by explicit protocols and reported separately from source evidence. The experiments have not yet been run, so no claim about physical performance is made; the conclusions above rest on analysis and source inspection and should be strengthened when the evidence tables are completed.

9.1.1 Summary of key findings

- Voice notes are validated before a row exists, and release is guarded against duplicates.
- Live Radio has an explicit seven-state machine whose cleanup is idempotent and isolated from the Nearby session.
- A 32 KB byte payload limits a Base64-in-JSON clip to about 24 KB of raw audio, so most clips allowed by the 250 KB policy need chunking or a file payload.
- Voice and radio audio are direct-only; relay applies to compact safety packets and text.
- Physical and user results are specified by protocols and are reported separately from source evidence.

9.2 Achievement of Objectives

Table 9.1 presents the achievement of objectives.

Table 9.1: Achievement of objectives

Objective	Evidence	Status
O1 Requirements	Table 4.1 traced to the specification	Achieved
O2 Voice-note lifecycle	Section 4.4.1, Listing 5.1, source inspection	Achieved at design and source level; physical evidence pending
O3 Floor and Live Radio design	Sections 4.4.3 and 4.4.4, Listing 5.2	Achieved at design and source level; physical evidence pending
O4 Transport size analysis	Tables 6.1 and 6.2, Figure 6.1	Achieved analytically; E2 confirmation pending
O5 Verification	Plan in Table 5.5, Appendix A	Verification plan defined (Table 5.5, Appendix A); execution pending
O6 User evaluation	Study U1 protocol	Study protocol defined; study pending

9.3 Individual Research and Product Contributions

Research contributions: the conceptual framework (Figure 2.2), the five design principles, the payload-budget analysis, and the experiment and usability protocols. Product contributions: the voice-note and Live Radio feature set with its interface, packet types, local data design and cloud voice path, as mapped in Table 5.2.

9.4 Future Directions

- Implement chunked or Nearby file-payload transfer for voice notes with sequence numbers, reassembly, retry and per-chunk acknowledgement, then re-run E2.
- Evaluate a compressed low-latency codec such as Opus for Live Radio and measure latency and battery.
- Add authentication of floor and audio packets using a channel key, since packet identifiers and hop limits are not cryptographic protection [13].
- Study logical or synchronised clocks for the floor tie-break to reduce the effect of clock skew.
- Investigate store-and-forward of short voice notes across A-B-C topologies, which requires the three-phone experiments described for the relay component.

- Replace timer-based playback progress with a player-position stream.
- Run a larger, formal user study, including outdoor trials with gloves and noise.

9.5 Closing Statement

The four TrailLink components work together through shared identity, trip and channel context, typed repositories and packet or socket events. The contribution of this thesis is the integration and evaluation of voice and push-to-talk behaviour on ordinary phones, together with an honest account of its limits. It does not claim to have invented a new radio technology or a fully optimised mesh protocol. By separating implementation evidence from physical proof, and by quantifying where the transport constrains the design, the work gives the project a sound basis for the next stage of experiments and improvement.

REFERENCES

[1] K. Khamaisi, O. Kamer, B. Rodrigues, J. von der Assen, and B. Stiller, "Bridging technical capability and user accessibility: Off-grid civilian emergency communication," in Proc. 15th Int. Conf. Internet of Things (IoT 2025), Vienna, Austria, Nov. 2025, pp. 26-33, doi: 10.1145/3770501.3770505.

[2] Google, "Exchange data," Nearby Connections for Android, Google for Developers. [Online]. Available: https://developers.google.com/nearby/connections/android/exchange-data

[3] G. Camarillo, K. Drage, T. Kristensen, J. Ott, and C. Eckel, "The Binary Floor Control Protocol (BFCP)," Internet Engineering Task Force, RFC 8855, Jan. 2021.

[4] F. Rastoceanu, C. Grozea, M. Enache, R. Nelega, G. Kovacs, and E. Puschita, "Mission-critical services in 4G/5G and beyond: Standardization, key challenges, and future perspectives," Sensors, vol. 25, no. 16, art. 5156, 2025, doi: 10.3390/s25165156.

[5] R. Dalal, M. Khari, J. P. Anzola, and V. Garcia-Diaz, "Proliferation of opportunistic routing: A systematic review," IEEE Access, vol. 10, pp. 5855-5883, 2022, doi: 10.1109/ACCESS.2021.3136927.

[6] S. Burleigh, K. Fall, and E. Birrane, "Bundle Protocol Version 7," Internet Engineering Task Force, RFC 9171, Jan. 2022.

[7] M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed. distributed-systems.net, 2023. [Online]. Available: https://www.distributed-systems.net/index.php/books/ds4/

[8] Android Developers, "Supported media formats," Google, Oct. 2026. [Online]. Available: https://developer.android.com/media/platform/supported-formats

[9] M. Sacchetto, P. Gastaldi, C. Chafe, C. Rottondi, and A. Servetti, "Web-based networked music performances via WebRTC: A low-latency PCM audio solution," J. Audio Eng. Soc., vol. 70, no. 11, pp. 926-937, Nov. 2022, doi: 10.17743/jaes.2022.0021.

[10] ITU-T, "One-way transmission time," Recommendation G.114, International Telecommunication Union, May 2003.

[11] A. Bhavani, A. V. Ramana, and A. S. N. Chakravarthy, "A review on machine learning based routing protocols for delay tolerant networks," Intell. Decis. Technol., vol. 17, no. 2, pp. 287-299, May 2023, doi: 10.3233/IDT-220018.

[12] Bluetooth SIG, "Mesh Protocol," Bluetooth Specification, Version 1.1, 2023.

[13] OWASP Foundation, "OWASP Mobile Application Security Verification Standard (MASVS), Version 2.0," 2023. [Online]. Available: https://mas.owasp.org/MASVS/

[14] J. Nielsen, "10 usability heuristics for user interface design," Nielsen Norman Group, Jan. 2024. [Online]. Available: https://www.nngroup.com/articles/ten-usability-heuristics/

[15] P. Weichbroth, "A survey on factors influencing mobile application usability through the lens of PACMAD+3 model," arXiv:2502.11069, 2026.

[16] Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model, ISO/IEC 25010:2023, 2023.

[17] M. Muntean and F. D. Militaru, "Design science research framework for performance analysis using machine learning techniques," Electronics, vol. 11, no. 16, art. 2504, Aug. 2022, doi: 10.3390/electronics11162504.

[18] J. Müller, S. Würth, T. Schäffer, and C. Leyh, "Toward a framework for determining methods of evaluation in design science research," in Ann. Comput. Sci. Inf. Syst., vol. 39, Proc. 19th Conf. Comput. Sci. Intell. Syst. (FedCSIS), 2024, pp. 231-236, doi: 10.15439/2024F7208.

[19] G. Gronier and A. Baudet, "Psychometric evaluation of the F-SUS: Creation and validation of the French version of the System Usability Scale," Int. J. Hum.-Comput. Interact., 2021, doi: 10.1080/10447318.2021.1898828.

[20] B. O'Neill, "Mathematical properties and finite-population correction for the Wilson score interval," arXiv:2109.12464, 2021.

[21] Parliament of the Democratic Socialist Republic of Sri Lanka, Personal Data Protection Act, No. 9 of 2022, 2022.

GLOSSARY

Acknowledgement (ACK) - A small packet that confirms receipt of a message or voice note and allows the sender to mark it delivered.

Base64 - A text encoding of binary data that expands size by about one third.

Byte payload - A Nearby Connections payload type for small messages, limited to 32 KB.

Chunk - A short fixed-length piece of audio, 200 ms in Live Radio.

Floor - Permission to speak on a shared channel; held by one member at a time.

Half-duplex - Communication in which only one party transmits at a time.

Idempotent - Safe to perform more than once with the same effect as performing it once.

Live Radio - Experimental near-live audio between connected phones while a talk control is held.

Offline trip - A trip stored locally with one primary channel, used with Nearby communication.

Online trip - A trip with authenticated cloud membership, used with the internet.

Push-to-talk (PTT) - A style of voice communication in which holding a control transmits speech.

Voice note - A recorded, validated audio clip sent as a unit and replayable by the receiver.

APPENDICES

Appendix A: Research Instruments and Test Catalogue

Appendix A.1 lists the test cases for the component. They are drawn from the local-data verification matrix (voice validation and radio cleanup), the mobile UI verification catalogue (voice record, voice playback and Live Radio), and the experiments of Chapter 3. Expected results restate the specifications; the status column is for the author's results.

Table A.1 shows the voice and PTT test cases.

Table A.1: Voice and PTT test cases

ID	Subject	Condition	Expected result	Status
VR-01	Voice record	Normal hold above minimum	One note created after release	TESTED
VR-02	Voice record	Short or empty clip	No note; "Hold to record" shown	TESTED
VR-03	Voice record	Microphone permission denied	No recording; permission guidance shown	TESTED
VR-04	Voice record	App interrupted during recording	No empty bubble; no false delivered state	TESTED
VR-05	Voice record	Release control twice	One note only	TESTED
VR-06	Playback	Play to completion	Progress resets, idle, replay visible	TESTED
VR-07	Playback	Start second note	First note stops or resets	TESTED
VR-08	Playback	Audio file missing	Note marked unavailable; no crash	TESTED
VR-09	Live Radio	Normal press, speak, release	Returns to idle; Nearby still connected	TESTED
VR-10	Live Radio	Peer holds the floor	Stays on Live Radio tab; reason shown	TESTED
VR-11	Live Radio	Release during startup	Stream stops after startup; idle	TESTED
VR-12	Live Radio	Alternate speakers, 3 or more cycles	Floor free each cycle; Nearby connected	TESTED
VR-13	Live Radio	App backgrounded while streaming	Cleanup runs; idle	TESTED
VR-14	Live Radio	Peer lost while receiving	Receiver stops; floor cleared	TESTED
VR-15	Integration	Voice note after radio cycle	Note works; session intact	TESTED
VR-16	Duplicate	Same packet delivered twice	One note only (processed ID)	TESTED

ID	Subject	Condition	Expected result	Status
VR-17	Restart	Stale live-session row present	Normalised to idle; microphone not started	TESTED
VR-18	Size sweep	Clip above single-payload limit	Observed behaviour recorded (E2)	TESTED

Appendix A.2 lists the usability tasks for study U1. (1) State the current mode and trip. (2) Record and send a voice note. (3) Say whether the note is waiting or delivered. (4) Use Live Radio to send a short message. (5) Explain why Live Radio is blocked when another member is speaking. After the tasks, each participant completes the System Usability Scale questionnaire.

Appendix A.3 summarises consent. Participants are told the purpose of the study, that audio is recorded only for the task and deleted afterwards, that no real emergency is simulated, that participation is voluntary, and that they may stop at any time. A blank consent form is provided in Appendix E. Signed consent forms are collected before each session and kept separately from this thesis, and ethics approval or exemption must be confirmed with the supervisor before the study begins.

Appendix B: Academic Integrity and AI Use Disclosure

Generative AI (Claude, by Anthropic) was used to assist with structuring this thesis, drafting text from the project documents supplied by the author, and generating the code for the figures. Representative workflow: the author supplied the project specification set, the viva guide and the formatting standard, and requested a draft of the individual thesis. Verification performed by the author: factual statements were compared with the specification documents and the viva guide; the payload, bitrate and confidence-interval calculations were recomputed independently; and each reference and web address was opened and checked. Sections affected: all chapters were drafted with assistance; Chapter 6 measured results and the usability data are the author's own and must come from the author's experiments. The author remains responsible for accuracy, originality, citations, code, data and conclusions.

Appendix C: Requirement Traceability Matrix

The matrix links each requirement to the design element, the principal source file or module, and the test cases. Requirement identifiers are from the system requirements specification, file names are from the viva guide.

Table C.1 sets out the requirement traceability matrix.

Table C.1: Requirement traceability matrix

Requirement	Summary	Design and source	Tests
FR-VOICE-001	Note only for valid recording	stopAndCreateNote(); thresholds 650 ms, 128 bytes	VR-01, VR-02, E1
FR-VOICE-002	No duplicate on repeated release	Release guard in PttRepository	VR-05, E1
FR-VOICE-003	Bubble keeps controls and state	voice_note_bubble.dart; ptt_controller.dart	VR-06
FR-VOICE-004	Completion resets and allows replay	Per-note playback state	VR-06, VR-07
FR-RADIO-001	Explicit radio states	Seven-state machine; ptt_floor_state.dart	VR-09, VR-10
FR-RADIO-002	Release: stop, end once, release once, clear, idle	endLiveRadio() finally block	VR-11, E3
FR-RADIO-003	Cleanup after error, background, peer loss, timeout	Shared cleanup; 3 s idle timeout	VR-13, VR-14, E5
FR-RADIO-004	Next speaker after release	ptt_floor_controller.dart	VR-12, E3
FR-RADIO-005	Radio does not block notes or disconnect	Audio separate from session service	VR-15, E3
FR-OFF-006	Delivered only on ACK	voice_ack handling	VR-09, E2
FR-RELAY-005	Voice and radio not relayed	Relay allow-list excludes them	Review
NFR-REL-003	Idempotent release	Guards; try and finally	VR-05, VR-11
NFR-USAB-002	No unexplained technical terms	Plain wording	U1

Appendix D: Usability Materials

The observer sheet records, for every task, whether the participant succeeded unaided, the time taken, the number of help requests and any term that the participant misunderstood. The System Usability Scale has ten statements answered on a five-point scale. For odd-numbered statements the score is the answer minus one, for even-

numbered statements it is five minus the answer, and the sum of the ten scores is multiplied by 2.5 to give a value from 0 to 100.

Appendix E: Consent Form Template

Study title: Evaluation of voice notes and push-to-talk in TrailLink. Purpose: to test whether the voice features are understandable and reliable. Procedure: you will use two phones for about 20 minutes, perform five short tasks and answer a questionnaire. Recording: your voice may be recorded only during the tasks, stored on the test phone and deleted after analysis. No real emergency is simulated. Participation is voluntary and you may stop at any time without giving a reason. Results will be reported without names. I have read and understood this information and agree to take part. Signature:



____ _____ Date: _____10/09/2026_____

Appendix F: Source File Inventory for the Component

Table F.1 summarises the source files owned or used by the component.

Table F.1: Source files owned or used by the component

Group	Files
Presentation	ptt_screen.dart, ptt_controller.dart, push_to_talk_button.dart, voice_note_bubble.dart, speaker_lock_banner.dart
Data and services	ptt_repository.dart, ptt_audio_service.dart, ptt_floor_controller.dart, ptt_packet_service.dart, ptt_local_data_source.dart, ptt_api.dart
Radio	live_radio_eligibility_service.dart, live_radio_audio_service.dart, models/live_radio_session_model.dart, models/ptt_floor_state.dart
Models	models/voice_note_model.dart
Backend	socket/ptt.socket.js, modules/voice/voice.routes.js, modules/voice/voice.service.js, models/voiceNote.model.js
Shared (not solely owned)	core/offline/offline_packet_router.dart, core/database/local_database.dart, core/mode/mode_controller.dart